

PUC
RIO

Esse ai resolve com CCW

Thomaz Miranda, Miguel Batista, Joao Arthur Marques

August 24, 2026

Code Contents

1 Data Structures 1
- 1.1 Fenwick Tree with Range Updates . . . 1
- 1.2 Convex Hull Trick Dinamico . . . 1
- 1.3 Disjoint Set Union (Union-Find) . . . 1
- 1.4 DSU with Rollback (Persistent/Undoable Union-Find) . . . 1
- 1.5 Segment Tree Over Time (Dynamic Connectivity Skeleton) . . . 1
- 1.6 Fenwick Tree (Binary Indexed Tree) . . . 2
- 1.7 Custom Hash for hash table . . . 2
- 1.8 Mo's Algorithm (Offline Range Queries) . . . 2
- 1.9 Order-Statistic Tree (PBDS) . . . 2
- 1.10 Persistent Segment Tree - O(log n) per query/update . . . 2
- 1.11 SegTree 2D Iterativa . . . 3
- 1.12 Segment Tree (Range Query + Point Update) 3
- 1.13 Segment Tree (Range Query + Range Update) 3
- 1.14 Sparse Table (Idempotent Range Query) . 4
- 1.15 Implicit Treap (Sequence Treap) . . . 4

2 Graph Algorithms 4
- 2.1 2-SAT . . . 4
- 2.2 Articulation Points / Bridges / Biconnected Components . . . 5
- 2.3 Bellman-Ford Single-Source Shortest Paths 5
- 2.4 Centroid Decomposition - pares a distancia k 5
- 2.5 Centroid Tree . . . 5
- 2.6 Dijkstra's Shortest Paths . . . 6
- 2.7 Dinic's Maximum Flow (with Scaling) . . 6
- 2.8 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) . . . 6
- 2.9 Dominator Tree (Lengauer-Tarjan) . . . 7
- 2.10 Euler Path / Euler Cycle . . . 7
- 2.11 Floyd-Warshall Algorithm . . . 7
- 2.12 Heavy Light Decomposition (Path query + Path update) . . . 7
- 2.13 Hungarian algorithm — min-cost perfect matching . . . 8
- 2.14 LCA com Euler tour + Sparse Table (RMQ) 8
- 2.15 LCA com binary lifting . . . 8
- 2.16 Min Cost Max Flow (MCMF) . . . 8
- 2.17 Strongly Connected Components (Kosaraju) 9
- 2.18 Topological Sort (Kahn's Algorithm) . . . 9
- 2.19 Virtual Tree . . . 9

3 Mathematics 9
- 3.1 Chinese Remainder Theorem . . . 9
- 3.2 Extended Eucidian Algorithm . . . 10
- 3.3 FFT/NTT Convolution . . . 10
- 3.4 Divisor Decomposition / Square Root Trick 10
- 3.5 Gauss . . . 10
- 3.6 Gauss - Z2 . . . 10
- 3.7 Flattened Divisor Sieve (CSR format) . . . 11
- 3.8 Matrix template . . . 11
- 3.9 Miller-Rabin Primality Test (Deterministic for 64-bit) . . . 11
- 3.10 Modular Integer . . . 11
- 3.11 Modular Arithmetic Helpers . . . 12
- 3.12 Discrete Logarithm (Baby-step Giant-step) 12
- 3.13 Discrete Square Root . . . 12
- 3.14 Primitive Root (Generator) . . . 12
- 3.15 Quadratic Field Extension (Z_p[sqrt(D)]) . 12
- 3.16 Linear Sieve (prime precomputation) . . . 13

4 String Algorithms 13
- 4.1 Aho-Corasick Automaton . . . 13
- 4.2 Knuth-Morris-Pratt (KMP) . . . 13
- 4.3 Suffix Array - O(n log n) . . . 13
- 4.4 Trie (Prefix Tree) . . . 14
- 4.5 Z-funtion . . . 14

5 Dynamic Programming 14
- 5.1 Divide and Conquer DP Optimization . . . 14
- 5.2 Sum over Subsets DP (SOS DP) . . . 14

6 Computational Geometry 14
- 6.1 Circumcircle and Circumradius . . . 14
- 6.2 Convex Hull (Monotone Chain) . . . 14
- 6.3 Extremal Vertex in Convex Polygon . . . 15
- 6.4 Float Geometry Primitives . . . 15
- 6.5 Integer Geometry Primitives . . . 16
- 6.6 Halfplane Intersection . . . 16
- 6.7 Point Inside Polygon . . . 16
- 6.8 Point Inside Convex Polygon . . . 16
- 6.9 Minimum Enclosing Circle . . . 17
- 6.10 Minkowski Sum . . . 17
- 6.11 Plota Poligonos . . . 17
- 6.12 Polygon Cut . . . 17
- 6.13 Segment Sweep Line Skeleton . . . 17

7 Extra (Templates and Utilities) 17
- 7.1 bash_alias . . . 17
- 7.2 1D Coordinate Compression . . . 17
- 7.3 hash . . . 17
- 7.4 makefile . . . 17
- 7.5 Otimizacoes agressivas, pode deixar mais rapido ou mais devagar . . . 17
- 7.6 random . . . 17
- 7.7 stress . . . 17
- 7.8 template . . . 18
- 7.9 tester . . . 18

Theoretical Contents

1 Counting Problems 18
- 1.1 Burnside's Lemma . . . 18
- 1.2 Catalan Numbers . . . 18

2 Graphs 18
- 2.1 Sum of Subtrees of a Tree . . . 18

3 Number Theory 18
- 3.1 Chicken McNugget Theorem . . . 18
- 3.2 Identities . . . 19
- 3.3 Large Prime Gaps . . . 19
- 3.4 Lucas's Theorem . . . 19
- 3.5 Mobius Inversion . . . 19
- 3.6 Number of Divisors . . . 19
- 3.7 Prime Counting Function - $\pi(x)$ . . . 19
- 3.8 Some Primes . . . 19

1 Data Structures

1.1 Fenwick Tree with Range Updates

ad0

Implements a pair of BITs to support 0-based range add updates and range sum queries efficiently.

Complexity: O(log N) per op, O(N)

```
5aa class BIT{
3ba     ll bit[2][MAX+2];
4de     ll n;
673 public:
c2e     BIT(ll n2, vll& v) {
1e3         n = n2;
914         for (ll i = 1; i <= n; i++)
edd             bit[1][min(n+1, i+(i&-i))] += bit[1][i] += v[i-1];
d0c     }
16a     ll get(ll x, ll i) {
b73         ll ret = 0;
360         for (; i; i -= i&-i) ret += bit[x][i];
edf         return ret;
346     }
23b     void add(ll x, ll i, ll val) {
503         for (; i <= n; i += i&-i) bit[x][i] += val;
669     }
f6e     ll get2(ll p) {
c7c         return get(0, p) * p + get(1, p);
006     }
235     ll query(ll l, ll r) {
ff5         return get2(r+1) - get2(l);
e1d     }
ccd     void update(ll l, ll r, ll x) {
e5f         add(0, l+1, x), add(0, r+2, -x);
f58         add(1, l+1, -x*l), add(1, r+2, x*(r+1));
4b5     }
ad0 }
```

1.2 Convex Hull Trick Dinamico

5ae

para double, use INF = 1/.0, div(a, b) = a/b update(x) atualiza o ponto de intersecao da reta x overlap(x) verifica se a reta x sobrepoe a proxima add(a, b) adiciona reta da forma ax + b query(x) computa maximo de ax + b para entre as retas O(log(n)) amortizado por insercao O(log(n)) por query

```
72c struct Line {
073     mutable ll a, b, p;
8e3     bool operator<(const Line& o) const { return a < o.a; }
abf     bool operator<(ll x) const { return p < x; }
469 };

1b7 struct CHT : multiset<Line, less<>> {
33a     ll div(ll a, ll b) {
353         return a / b - ((a ^ b) < 0 && a % b);
10f     }

bbb     void update(iterator x) {
459         if (next(x) == end()) x->p = INF;
eec         else if (x->a == next(x)->a) x->p = x->b >= next(x)->b ?
 INF : -INF;
424         else x->p = div(next(x)->b - x->b, x->a - next(x)->a);
d37     }

71c     bool overlap(iterator x) {
f18         update(x);
cfa         if (next(x) == end()) return 0;
a4a         if (x->a == next(x)->a) return x->b >= next(x)->b;
d40         return x->p >= next(x)->p;
901     }

176     void add(ll a, ll b) {
1c7         auto x = insert({a, b, 0});
4ab         while (overlap(x)) erase(next(x)), update(x);
f29         if (x != begin() && !overlap(prev(x))) x = prev(x),
update(x);
bc7         while (x != begin() && overlap(prev(x)))
4d2             x = prev(x), erase(next(x)), update(x);
500     }
```

```
4ad     ll query(ll x) {
229         assert(!empty());
7d1         auto l = *lower_bound(x);
d41 #warning cuidado com overflow!
aba         return l.a * x + l.b;
3f5     }
e4b };
```

1.3 Disjoint Set Union (Union-Find) b72

Supports find with path compression and union by size to maintain dynamic connectivity of disjoint sets.

Complexity: O(alpha(N)) amortized per op, O(N)

```
8d3 struct dsu {
320     vll id, len;

947
    dsu(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(), 0); }

f21
    ll find(ll a) { return a == id[a] ? a : id[a] = find(id[a]); }

2fe     bool uni(ll a, ll b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
8d6         if (len[a] < len[b]) swap(a, b);
f5f         len[a] += len[b], id[b] = a;
8a6         return true;
9b2     }
b72 };
```

1.4 DSU with Rollback (Persistent/Undoable Union-Find) 4a6

Allows merging sets and undoing operations to a previous state using a stack of modifications. No Path Compression to keep tree structure reversible.

Complexity: O(log N) per find/unite, O(1) per checkpoint/rollback

```
4e8 struct dsu_rb {
320     vll id, len;
3c0     ll allbip = true;
a32     stack<pair<ll&, ll>> st;

9c0     dsu_rb(ll n) : id(n), len(n, 1) {
db8         iota(id.begin(), id.end(), 0);
391     }

18a     void save(ll &x) {  st.emplace(x, x);}

5ac     ll time() { return sz(st); }
3de     void rollback(ll t) {
21d         while(sz(st) > t) {
880             auto [end, val] = st.top(); st.pop();
149             end = val;
5bf         }
c7c     }

d8f     ll find(ll a) { return a == id[a] ? a : find(id[a]); }

040     bool unite(ll a, ll b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
8d6         if (len[a] < len[b]) swap(a, b);
f67         save(len[a]);
cba         save(id[b]);
f5f         len[a] += len[b], id[b] = a;
8a6         return true;
e7f     }
4a6 };
```

1.5 Segment Tree Over Time (Dynamic Connectivity Skeleton) bc7

Stores edge activation intervals in a segment tree over time to enable offline dynamic connectivity with rollback DSU.

```
120 struct time_query{
f88     ll l, r;
60d     time_query(ll l_, ll r_){
ae0         l = l_;
91d         r = r_;
920     }
ef5 };

d13 struct time_node {
ee4     ll lm, rm;
b4f     unique_ptr<time_node> lc, rc;

a22     vector<time_query> op;

da3     time_node(ll lm_, ll rm_){
a44         lm = lm_;
d79         rm = rm_;
be2         if (lm != rm) {
554             ll mid = (lm + rm) / 2;
d44             lc = make_unique<time_node>(lm, mid);
30e             rc = make_unique<time_node>(mid + 1, rm);
746         }
7f2     }

514     void add_query(ll lq, ll rq, time_query x) {
473         if (rq < lm || lq > rm) return;
9e3         if (lq <= lm && rm <= rq) {
488             op.push_back(x);
505             return;
335         }
455         lc->add_query(lq, rq, x);
a82         rc->add_query(lq, rq, x);
3d3     }
127 };
```

1.6 Fenwick Tree (Binary Indexed Tree) d0d

Supports point updates and prefix/range sum queries in logarithmic time using a 1-indexed BIT.

Complexity: O(log N) per op, O(N)

```
8eb struct Bit {
4de     ll n;
643     vll bit;
dd0     Bit(ll _n = 0) : n(_n), bit(n + 1) {}
cb1     Bit(vll& v) : n(sz(v)), bit(n + 1) {
518         for (ll i = 1; i <= n; i++) {
671             bit[i] += v[i - 1];
c8f             ll j = i + (i & -i);
b8a             if (j <= n) bit[j] += bit[i];
154         }
dc7     }
e55     void update(ll i, ll x) { // soma x na posicao i
b64         for (i++; i <= n; i += i & -i) bit[i] += x;
6f4     }
2c0     ll pref(ll i) { // soma [0, i]
b73         ll ret = 0;
4d3         for (i++; i; i -= i & -i) ret += bit[i];
edf         return ret;
af2     }
235     ll query(ll l, ll r) {  // soma [l, r]
89b         return pref(r) - pref(l - 1);
aa0     }
f46     ll upper_bound(ll x) {
62d         ll p = 0;
370         for (ll i = __lg(n); i+1; i--)
28b             if (p + (1<<i) <= n && bit[p + (1<<i)] <= x)
68e                 x -= bit[p += (1 << i)];
74e         return p;
052     }
d0d };
```

1.7 Custom Hash for hash table 629

Provides 64-bit hashers for integers and pairs to use with __gnu_pbds::gp_hash_table.

Complexity: O(1) average, O(n)

```
c4d #include <bits/extc++.h>

// for ll
75f struct chash {
5d6     const uint64_t C = ll(4e18 * acos(0)) | 71;
2cf     ll operator()(ll x) const { return __builtin_bswap64(x*C); }
cdd };

// for pll
75f struct chash {
232     size_t operator()(const pll& p) const {
cc9         return p.first ^ __builtin_bswap64(p.second);
ba6     }
384 };

b6a __gnu_pbds::gp_hash_table<ll, ll, chash> h({},{},{},{},{1<<16});
4ca
__gnu_pbds::gp_hash_table<pll, ll, chash> h({},{},{},{},{1<<16});
```

1.8 Mo's Algorithm (Offline Range Queries) e5f

Answers offline range queries by ordering them (block or Hilbert curve) to get small pointer movement and amortized updates.

Complexity: O((N + Q) sqrt N), O(N)

```
c41 const ll MAX = 2e5+10;
29b const ll SQ = sqrt(MAX);
1b0 ll ans;

fd9 inline void insert(ll p) {
7d3 }

155 inline void erase(ll p) {
027 }

280 inline ll hilbert(ll x, ll y) {
1ea     static ll N = 1 << (__builtin_clzll(0ll) - __builtin_clzll(
MAX));
5bc     ll rx, ry, s;
b72     ll d = 0;
43b     for (s = N/2; s > 0; s /= 2) {
```

```
        rx = (x & s) > 0, ry = (y & s) > 0;
        d += s * ll(s) * ((3 * rx) ^ ry);
        if (ry == 0) {
            if (rx == 1) x = N-1 - x, y = N-1 - y;
            swap(x, y);
        }
    }
    return d;
}

#define HILBERT true
vll MO(vector<pair<ll, ll>> &q) {
    ans = 0;
    ll m = sz(q);
    vll ord(m);
    iota(ord.begin(), ord.end(), 0);
#if HILBERT
    vll h(m);
    for (ll i = 0; i < m; i++) h[i] = hilbert(q[i].first, q[i].second);
    sort(ord.begin(), ord.end(), [&](ll l, ll r) { return h[l] < h[r]; });
#else
    sort(ord.begin(), ord.end(), [&](ll l, ll r) {
        if (q[l].first / SQ != q[r].first / SQ) return q[l].first < q[r].first;
        if ((q[l].first / SQ) % 2) return q[l].second > q[r].second;
        return q[l].second < q[r].second;
    });
#endif
    vll ret(m);
    ll l = 0, r = -1;

    for (ll i : ord) {
        ll ql, qr;
        tie(ql, qr) = q[i];
        while (r < qr) insert(++r);
        while (l > ql) insert(--l);
        while (l < ql) erase(l++);
        while (r > qr) erase(r--);
        ret[i] = ans;
    }
    return ret;
}
```

1.9 Order-Statistic Tree (PBDS) 398

Wraps __gnu_pbds tree to support order_of_key and find_by_order operations on a sorted set.

Complexity: O(log N) per op, O(N)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

#define ordered_set tree<pll, null_type,less<pll>, rb_tree_tag, tree_order_statistics_node_update>

int main() {
    ordered_set s;
    s.find_by_order(position);
    s.order_of_key(value);
}
```

1.10 Persistent Segment Tree - O(log n) per query/update 967

Supports point updates and range queries across versions

```
struct Node {
    ll val;
    ll l, r;
};

struct perseg {
    ll n, cnt = 0;
    vector<Node> nd;
    vll rt;

    perseg(vll& v, ll q = 0) : n(sz(v)) {
        nd.reserve(2*n + 18*(n+q));
        rt.reserve(q + 2);
        rt.push_back(cnt++);
        build(nd.data(), cnt, rt[0], 0, n-1, v);
    }

    static ll build(Node* d, ll& cnt, ll p, ll l, ll r, vll& v) {
        if (l == r) return d[p].val = v[l];
        d[p].l = cnt++; d[p].r = cnt++;
        ll m = (l+r)/2;
        return d[p].val = build(d, cnt, d[p].l, l, m, v) + build(d, cnt, d[p].r, m+1, r, v);
    }
    ll query(ll a, ll b, ll t) {
        return query(nd.data(), a, b, rt[t], 0, n-1);
    }
    static ll query(Node* d, ll a, ll b, ll p, ll l, ll r) {
        if (b < l || r < a) return 0;
        if (a <= l && r <= b) return d[p].val;
        ll m = (l+r)/2;
        return query(d, a, b, d[p].l, l, m) + query(d, a, b, d[p].r, m+1, r);
    }
    ll set(ll a, ll val, ll t) {
        ll nr = cnt++;
        set(nd.data(), cnt, a, val, rt[t], nr, 0, n-1);
        rt.push_back(nr);
        return (ll)rt.size()-1;
    }
    static ll set(Node* d, ll& cnt, ll a, ll val, ll lp, ll p, ll l, ll r) {
        if (l == r) return d[p].val = val;
        ll m = (l+r)/2;
        if (a <= m) {
            d[p].l = cnt++; d[p].r = d[lp].r;
            return d[p].val = set(d, cnt, a, val, d[lp].l, d[p].l, l, m) + d[d[p].r].val;
        }
        d[p].l = d[lp].l; d[p].r = cnt++;
        return d[p].val = d[d[p].l].val + set(d, cnt, a, val, d[lp].r, d[p].r, m+1, r);
    }
};
```

1.11 SegTree 2D Iterativa d51

Consultas 0-based Um valor inicial em (x, y) deve ser colocado em seg[x+n][y+n] Query: soma do retangulo ((x1, y1), (x2, y2)) Update: muda o valor da posicao (x, y) para val Nao pergunte como que essa coisa funciona Para query com distancia de manhattan <= d, faca nx = x+y, ny = x-y Update em (nx, ny), query em ((nx-d, ny-d), (nx+d, ny+d)) Se for de min/max, pode tirar os if's da 'query', e fazer sempre as 4 operacoes. Fica mais rapido Complexidades: build - O(n{}2) query - O(log{}2(n)) update - O(log{}2(n))

```
struct seg2d{
    ll n;
    vector<vll> seg;

    seg2d(ll n_, vector<vll>& init) : n(n_), seg(2 * (n+1), vll(2 * (n+1), 0)) {
        forn(i,0,n) forn(j,0,n)
            seg[i + n][j + n] = init[i][j];
        for (ll x = 2*n; x; x--) for (ll y = 2*n; y; y--) {
            if (x < n) seg[x][y] = seg[2*x][y] + seg[2*x+1][y];
            if (y < n) seg[x][y] = seg[x][2*y] + seg[x][2*y+1];
        }
    }

    ll query(ll x1, ll y1, ll x2, ll y2) {
        ll ret = 0, y3 = y1 + n, y4 = y2 + n;
        for (x1 += n, x2 += n; x1 <= x2; ++x1 /= 2, --x2 /= 2)
        for (y1 = y3, y2 = y4; y1 <= y2; ++y1 /= 2, --y2 /= 2) {
            if (x1%2 == 1 && y1%2 == 1) ret += seg[x1][y1];
            if (x1%2 == 1 && y2%2 == 0) ret += seg[x1][y2];
            if (x2%2 == 0 && y1%2 == 1) ret += seg[x2][y1];
            if (x2%2 == 0 && y2%2 == 0) ret += seg[x2][y2];
        }
        return ret;
    }

    void update(ll x, ll y, ll val) {
        ll y2 = y += n;
        for (x += n; x; x /= 2, y = y2) {
            if (x >= n) seg[x][y] = val;
            else seg[x][y] = seg[2*x][y] + seg[2*x+1][y];

            while (y /= 2) seg[x][y] = seg[x][2*y] + seg[x][2*y+1];
        }
    }
};
```

1.12 Segment Tree (Range Query + Point Update) 2be

supports point updates and range queries.

Complexity: O(log N) per op, O(N)

```
struct node {
    ll val = 0;

    static node comb(const node& a, const node& b) {
        return {a.val + b.val};
    }
};

template<typename T> struct segtree {
  ll n;
  T neutral;
  vector<T> tree;
```

```
b80   segtree(ll _n, T _neutral = T()) {
7b9     n = _n, neutral = _neutral;
aa3     tree.resize(2*n+1, neutral);
a98   }

dc3   void set_leaves(vector<T> &leaves) {
4f7     copy(leaves.begin(), leaves.end(), tree.begin() + n);

6e6     for (ll i = n - 1; i > 0; i--) tree[i] = T::comb(tree[2 * i],
 tree[2 * i + 1]);
144   }

418   void update(ll i, T v) {
83b     i += n;
11e     tree[i] = v;
b8f     while (i > 1) {
011       i /= 2;
00e       tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
13c     }
cec   }

0fc   T query(ll i, ll j) {
3d8     T rl = neutral, rr = neutral;
75a     for(i += n, j += n; i <= j; i /= 2, j /= 2){
bf3       if((i&1) == 1) rl = T::comb(rl, tree[i++]);
ebf       if((j&1) == 0) rr = T::comb(tree[j--], rr);
2ec     }
814     return T::comb(rl, rr);
600   }
3a3 };
```

1.13 Segment Tree (Range Query + Range Update) 5f6

Tree for range queries with a customizable combine; supports range updates and range queries.

Complexity: O(log N) per op, O(N) to build

```
bf2 struct Node {
325     ll sum, cnt;
298     Node operator*(const Node &o) const { return {sum + o.sum,
cnt + o.cnt}; }
81b };

820 struct Update {
8bb     ll add = 0;
c81     optional<ll> set;

e0b     Node operator()(const Node &n) const {
49b         ll res = set.has_value() ? *set * n.cnt : n.sum;
9dd         return {res + add * n.cnt, n.cnt};
bf2     }

487     Update operator+(const Update &o) const {
ae9         Update res = *this;
035         if (o.set.has_value()) {
5ad             res.set = o.set;
d0d             res.add = 0;
902         }
7fc         if (o.add != 0) {
354             if (res.set.has_value()) *res.set += o.add;
ade             else res.add += o.add;
818         }
b50         return res;
b1d     }
d61 };

188 template<typename T, typename U> struct segtree {
0b8   ll s, h;

d0f   T id;
226   vector<T> val;

11c   U noop;
4af   vector<bool> dirty;
578   vector<U> prop;

3d6   segtree(ll ts, T tid = T(), U tnoop = U()) {
180     id = tid, noop = tnoop;
85c     for (s = 1, h = 1; s < ts; ) s *= 2, h++;

173     val.assign(2*s, id);
fa8     dirty.assign(2*s, false);
b4f     prop.assign(2*s, noop);
8d4   }

cb1   void set_leaves(vector<T> &lvs) {
e2d     copy(lvs.begin(), lvs.end(), val.begin() + s);

8b0     for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 *
 i + 1];
fa8     dirty.assign(2*s, false);
b4f     prop.assign(2*s, noop);
d52   }

732   void apply(ll i, U &upd) {
0ef     val[i] = upd(val[i]);
a35     if(i < s) {
083       prop[i] = prop[i] + upd;
deb       dirty[i] = true;
9df     }
af7   }

ae6   void pull(ll i) {
33b     for (ll l = i/2; l; l /= 2) {
218       T comb = val[2*l] * val[2*l+1];
5ba       val[l] = prop[l](comb);
e5c     }
aa8   }

cb0   void push(ll i) {
013     for (ll th = h; th > 0; th--) {
7a0       ll l = i >> th;

2fa       if (dirty[l]) {
02d         apply(2*l, prop[l]);
aed         apply(2*l+1, prop[l]);

62d         prop[l] = noop;
f99         dirty[l] = false;
ed9       }
dc3     }
381   }

98b   void update(ll i, ll j, U upd) {
5ba     i += s, j += s;
f44     push(i), push(j);
689     for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
b7e       if((l&1) == 1) apply(l++, upd);
b0a       if((r&1) == 0) apply(r--, upd);
30c     }

999     pull(i), pull(j);
fda   }

0fc   T query(ll i, ll j){
5ba     i += s, j += s;
f44     push(i), push(j);

494     T rl = id, rr = id;
a82     for(; i <= j; i /= 2, j /= 2){
a29       if((i&1) == 1) rl = rl * val[i++];
3dd       if((j&1) == 0) rr = val[j--] * rr;
e07     }
f7e     return rl * rr;
62e   }
2b0 };
```

1.14 Sparse Table (Idempotent Range Query) 9ba

Preprocesses static array to answer idempotent range queries (e.g., min/max) in O(1) after O(N log N) build.

Complexity: O(N log N) build, O(1) query; O(N log N) space

```
991 struct sparse {
815     vector<vll> m;

b55     sparse(vll& v) {
dce         ll n = sz(v);
eb5         ll logn = 64 - __builtin_clzll(n - 1);
2fa         m.resize(n+1, vll(logn+1));

7c9         forn(i, 0, n) m[i][0] = v[i];

aae         for (ll j = 1; (1 << j) <= n; j++)
82d         for (ll i = 0; i + (1 << j) <= n; i++)
631             m[i][j] = min(m[i][j-1], m[i + (1 << (j-1))][j-1]);
2a8     }

4de     ll query(ll a, ll b) {
b44         ll j = __builtin_clzll(1) - __builtin_clzll(b - a + 1);
a6f         return min(m[a][j], m[b - (1 << j) + 1][j]);
992     }
9ba };
```

1.15 Implicit Treap (Sequence Treap) 2a5

Maintains a sequence with split and merge operations using randomized priorities and subtree sizes.

Complexity: O(log N) expected per op, O(N)

```
125 struct Treap{
348     ll val;
0ce     ll prio, size;
330     vector<Treap*> kids;
b02     Treap(ll c): val(c), prio(rand()), size(1),
680         kids({NULL,NULL}){};
494 };

464 ll size(Treap *me){return me ? me->size : 0;}
e86 void rsz(Treap* me){me -> size =
```

```
24d     1 + size(me->kids[0]) + size(me->kids[1]);}

e8f vector<Treap*> split(Treap *me, ll idx){
878     if(!me) return {NULL,NULL};
032     vector<Treap*> out;

52a     if(size(me->kids[0]) < idx){
e1c         auto aux = split(me->kids[1],
312             idx - size(me->kids[0]) -1);
409         me->kids[1] = aux[0];
b14         rsz(me);
abb         out = {me, aux[1]};
aaa     }else{
c8a         auto aux = split(me->kids[0], idx);
c89         me->kids[0] = aux[1];
b14         rsz(me);
3cb         out = {aux[0], me};
d61     }
fe8     return out;
e7d }

b85 Treap* merge(Treap *left, Treap *right){
c10     if(left == NULL) return right;
096     if(right == NULL) return left;

671     Treap* out;

38d     if(left->prio < right->prio){
d90         left->kids[1] = merge(left->kids[1], right);
122         rsz(left);
d7a         out = left;
bbb     }else{
cea         right->kids[0] = merge(left, right->kids[0]);
e85         rsz(right);
015         out = right;
2f1     }
fe8     return out;
499 }
```

2 Graph Algorithms

2.1 2-SAT

041

solve() retorna um par, o first fala se eh possivel atribuir, o second fala se cada variavel eh verdadeira O(|V|+|E|) = O(#variaveis + #restricoes)

```
138 struct sat {
446     ll n, tot;
838     vector<vll> g;
9ba     vll vis, comp, id, ans;
d1b     vll s;

141     sat() {}
5f5     sat(ll n_) : n(n_), tot(n), g(2*n) {}

81c     ll dfs(ll i, ll& t) {
02f         ll lo = id[i] = t++;
c3f         s.push_back(i), vis[i] = 2;
9c7         for (ll j : g[i]) {
740             if (!vis[j]) lo = min(lo, dfs(j, t));
994             else if (vis[j] == 2) lo = min(lo, id[j]);
3aa         }
3de         if (lo == id[i]) while (1) {
b8e             ll u = s.back(); s.pop_back();
9c5             vis[u] = 1, comp[u] = i;
121             if ((u>>1) < n && ans[u>>1] == -1) ans[u>>1] = ~u&1;
2ef             if (u == i) break;
9fd         }
253         return lo;
c25     }

286     void add_impl(ll x, ll y) { // x -> y = !x ou y
26a         x = x >= 0 ? 2*x : -2*x-1;
2b8         y = y >= 0 ? 2*y : -2*y-1;
a1e         g[x].push_back(y);
1e2         g[y^1].push_back(x^1);
677     }
9b5     void add_cl(ll x, ll y) { // x ou y
0b5         add_impl(~x, y);
135     }
37f     void add_xor(ll x, ll y) { // x xor y
0b7         add_cl(x, y), add_cl(~x, ~y);
4ac     }
680     void add_eq(ll x, ll y) { // x = y
c86         add_xor(~x, y);
9b1     }
874     void add_true(ll x) { // x = T
18b         add_impl(~x, x);
ea8     }
44e     void at_most_one(vll v) { // no max um verdadeiro
051         g.resize(2*(tot+sz(v)));
758         for (ll i = 0; i < sz(v); i++) {
8c9             add_impl(tot+i, ~v[i]);
a8f             if (i) {
b6a                 add_impl(tot+i, tot+i-1);
3d3                 add_impl(v[i], tot+i-1);
0f7             }
691         }
253         tot += sz(v);
202     }

806     pair<bool, vll> solve() {
c79         ans = vll(n, -1);
b1b         ll t = 0;
449         vis = comp = id = vll(2*tot, 0);
fe3         for (ll i = 0; i < 2*tot; i++) if (!vis[i]) dfs(i, t);
d99         for (ll i = 0; i < tot; i++)
4c9             if (comp[2*i] == comp[2*i+1]) return {false, {}};
997         return {true, ans};
4b3     }
041 };
```

2.2 Articulation Points / Bridges / Biconnected Components

520

is_art[i] = true se i eh ponto de articulacao (ART) bridges: lista de arestas ponte (BRIDGE) bcc: lista de componentes biconexas (em arestas) (BCC) O(n+m)

```
f08 struct TJ {
26a     ll n, timer=0;
cb0     vector<vll>& g;
fc9     vll tin, low;

f15     vector<bool> is_art; // ART
5f2     vector<pll> bridges; // BRIDGE
950     vector<pll> st; // BCC
f5e     vector<vector<pll>> bcc; // BCC

1fc     TJ(ll n, vector<vll>& g): n(n), g(g){
cfd         tin.assign(n,-1);
af8         low.resize(n);
1a1         is_art.assign(n, false); // ART
4bc         forn(i, 0, n) if(tin[i]==-1) dfs(i);
56c     }

b6d     void dfs(ll v, ll p=-1){
f3d         tin[v]=low[v]=timer++;
8a8         ll ch=0;

72c         for(ll to:g[v]){
756             if(to==p) continue;

a5f             if(tin[to]==-1){
df6                 st.push_back({v,to}); // BCC

a72                 dfs(to,v);
575                 low[v] = min(low[v], low[to]);

f03                 if(low[to]>tin[v]) bridges.push_back({v,to}); //
 BRIDGE
88a                 if(low[to]>=tin[v] && p!=-1) is_art[v] = true; //
 ART

6aa                 if(low[to]>=tin[v]){ // BCC
cc9                   vector<pll> c; // BCC
31e                   while(1){ // BCC
69d                     auto e=st.back(); st.pop_back(); // BCC
d5e                     c.push_back(e); // BCC
ffa                     if(e.first==v && e.second==to) break; // BCC
cd6                   } // BCC
fc8                   bcc.push_back(c); // BCC
155                 } // BCC

dfc                 ch++;
0a9             }else{
6de                 low[v]=min(low[v],tin[to]);
328
                }
498             if(tin[to]<tin[v]) st.push_back({v,to});   // BCC
c38         }
30a         if(p==-1 && ch>1) is_art[v] = true;   // ART
b58     }
520 };
```

2.3 Bellman-Ford Single-Source Shortest Paths

a21

Computes shortest paths from s on directed graphs with possibly negative edge weights. Nodes reachable via a negative cycle get dist = -INF.

Complexity: O(N*M), O(N + M)

```
e7b void bellman_ford(ll s, ll n, vll& d, vll& p, vector<tuple<ll,ll,
ll>> edges) {
355     d.assign(n, INF);
d8d     p.assign(n, -1);
d66     d[s] = 0;

67e     sort(edges.begin(), edges.end(), [](auto& a, auto& b) {
596         ll sa = get<0>(a) < get<1>(a) ? get<0>(a) : -get<0>(a);
7ff         ll sb = get<0>(b) < get<1>(b) ? get<0>(b) : -get<0>(b);
0ac         return sa < sb;
3f1     });

a46     ll lim = n / 2 + 2; // use n-1 with unsorted edges
```

```
0bc     forn(i, 0, lim) for (auto [u, v, w] : edges) {
0e5         if (d[u] == INF || d[u] == -INF) continue;
86b         ll nd = d[u] + w;
61b         if (nd < d[v]) {
e42             p[v] = u;
3f9             d[v] = (i < lim - 1 ? nd : -INF);
db4         }
804     }
4d8     forn(i, 0, lim) for (auto [u, v, w] : edges)
a8f         if (d[u] == -INF) d[v] = -INF;
a21 }
```

2.4 Centroid Decomposition - pares a distancia k 9a2

Objetivo: contar #pares (u,v) com dist(u,v)=k na arvore. Ideia: em cada centroide c, para cada subarvore j: - path = profundidades d partindo de j (sem nos removidos) - somar cnt[k-d-1] ANTES de atualizar (evita dupla contagem entre subarvores) - depois, cnt[d+1]++ (a aresta (c,j) soma 1) Funcoes: dfs_sz (tamanhos), centroid (acha centroide), dfs (coleta depths), decomp (divide e combina). Complexidade: O(n log n) tempo; O(n) memoria. Cuidados: limites de cnt, k==0 trata a parte se quiser contar (u,u); rem[c] permanece marcado. escrito pelo gpt, cuidado

```
1fd const ll MAX = 100'005;
32a vll g[MAX];
a49 ll siz[MAX], rem[MAX];

945 void dfs(vll& path, ll i, ll l=-1, ll d=0) {
547     path.push_back(d);
113
    for (ll j : g[i]) if (j != l && !rem[j]) dfs(path, j, i, d+1);
3ee }

499 ll dfs_sz(ll i, ll l=-1) {
ae3     siz[i] = 1;
cac
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
c61     return siz[i];
79e }

c46 ll centroid(ll i, ll l, ll size) {
9fe
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
735         return centroid(j, i, size);
d9a     return i;
d5e }

27a ll decomp(ll i, ll k) {
79c     ll c = centroid(i, i, dfs_sz(i));
a67     rem[c] = 1;

     // gasta O(n) aqui - dfs sem ir pros caras removidos
04b     ll ans = 0;
8e1     vll cnt(siz[i]);
878     cnt[0] = 1;
e65     for (ll j : g[c]) if (!rem[j]) {
a0e         vll path;
baf         dfs(path, j);
ed7         for (ll d : path) if (0 <= k-d-1 && k-d-1 < siz[i])
285             ans += cnt[k-d-1];
477         for (ll d : path) cnt[d+1]++;
035     }

ffb     for (ll j : g[c]) if (!rem[j]) ans += decomp(j, k);
3f1     rem[c] = 0;
ba7     return ans;
de1 }
```

2.5 Centroid Tree a22

Constroi a centroid tree p[i] eh o pai de i na centroid-tree dist[i][k] = distancia na arvore original entre i e o k-esimo ancestral na arvore da centroid O(n log(n)) de tempo e memoria

```
1fd const ll MAX = 100'005;
70d vector<vll> g(MAX), dist(MAX);
d2b vll siz(MAX), rem(MAX), p(MAX);

499 ll dfs_sz(ll i, ll l=-1) {
ae3     siz[i] = 1;
cac
    for (ll j : g[i]) if (j != l && !rem[j]) siz[i] += dfs_sz(j, i);
c61     return siz[i];
79e }

c46 ll centroid(ll i, ll l, ll size) {
9fe
    for (ll j : g[i]) if (j != l && !rem[j] && siz[j] > size / 2)
735         return centroid(j, i, size);
d9a     return i;
d5e }

3de void dfs_dist(ll i, ll l, ll d=0) {
541     dist[i].push_back(d);
1b0     for (ll j : g[i]) if (j != l && !rem[j])
82a         dfs_dist(j, i, d+1);
225 }

457 void decomp(ll i, ll l = -1) {
79c     ll c = centroid(i, i, dfs_sz(i));
1b9     rem[c] = 1, p[c] = l;
534     dfs_dist(c, c);
1ef     for (ll j : g[c]) if (!rem[j]) decomp(j, c);
f75 }

145 void build(ll n) {
b26     forn(i,0,n) rem[i] = 0, dist[i].clear();
867     decomp(0);
40c     forn(i,0,n) reverse(dist[i].begin(), dist[i].end());
9d9 }
```

2.6 Dijkstra's Shortest Paths 699

Computes single-source shortest paths on non-negative weighted graphs using a priority queue.

Complexity: O((N + M) log N), O(N + M)

```
965 void dijkstra(ll s, vll &d, vll& p, vector<vector<pll>>& g) {
5b8     ll n = sz(g);
355     d.assign(n, INF);
d8d     p.assign(n, -1);

d66     d[s] = 0;
924     priority_queue<pll> pq;
7ba     pq.push({0, s});
502     while (!pq.empty()) {
5cd         ll u = pq.top().second;
6fd         ll d_u = -pq.top().first;
716         pq.pop();
211         if (d_u != d[u]) continue;

9c3         for (auto [v, w_v] : g[u]) {
f35             if (d[u] + w_v < d[v]) {
7ca                 d[v] = d[u] + w_v;
e42                 p[v] = u;
2a6                 pq.push({-d[v], v});
e72             }
48a         }
e88     }
699 }
```

2.7 Dinic's Maximum Flow (with Scaling) b72

Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels.

Complexity: O(E V$\hat{}$2) worst-case, O(E)

```
472 struct dinitz {
f61     const bool scaling;
d74     ll lim;
670     struct edge {
283         ll to, cap, rev, flow;
7f9         bool res;
764         edge(ll to_, ll cap_, ll rev_, bool res_)
a94
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
eb4     };

002     vector<vector<edge>> g;
697     vll lev, beg;
a71     ll F;
9da
    dinitz(ll n, bool scaling_) : scaling(scaling_), g(n), F(0){}

f3e     void add(ll a, ll b, ll c) {
de2         g[a].emplace_back(b, c, sz(g[b]), false);
a1d         g[b].emplace_back(a, 0, sz(g[a])-1, true);
3d3     }

6d8     bool bfs(ll s, ll t) {
f09         lev = vll(sz(g), -1); lev[s] = 0;
abd         beg = vll(sz(g), 0);
7a6         queue<ll> q; q.push(s);
647         while (sz(q)) {
c79             ll u = q.front(); q.pop();
bd9             for (auto& i : g[u]) {
e4f
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
17e                 if (scaling && i.cap - i.flow < lim) continue;
185                 lev[i.to] = lev[u] + 1;
8ca                 q.push(i.to);
00d             }
823         }
0de         return lev[t] != -1;
d7c     }

b95     ll dfs(ll v, ll t, ll f = INF) {
3e6         if (!f || v == t) return f;
78b         for (ll& i = beg[v]; i < sz(g[v]); i++) {
027             auto& e = g[v][i];
206             if (lev[e.to] != lev[v] + 1) continue;
934             ll foi = dfs(e.to, t, min(f, e.cap - e.flow));
749             if (!foi) continue;
```

```
3c5             e.flow += foi, g[e.to][e.rev].flow -= foi;
45c             return foi;
862         }
bb3         return 0;
468     }

074     ll max_flow(ll s, ll t) {
997         for (lim = scaling ? (1ll<<30) : 1; lim; lim /= 2)
69c             while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
4ff         return F;
a5d     }

f47     vector<pll> min_cut() {
956         vector<pll> cut;
21f         forn(u, 0, sz(g))
162             if (lev[u] != -1)
d4c                 for (auto& e : g[u])
554                     if (lev[e.to] == -1 && !e.res)
13d                         cut.push_back({u, e.to});
d1b         return cut;
667     }

e30     void reset() {
59f         F = 0;
843         for (auto& edges : g) for (auto& e : edges) e.flow = 0;
5d0     }
b72 };
```

2.8 Directed Minimum Spanning Tree (Edmonds / Chu-Liu) 7a8

finds the minimum cost arborescence rooted at src in a weighted directed graph. returns {cost, parent} where parent[u] is the predecessor of u in the optimal tree. returns {-1, {}} if no arborescence exists.

Complexity: O(M log N), O(N+M) me

```
43b #include "src/structures/dsu_rb.cpp"

d44 struct Edge {ll a, b, w; };
bf2 struct Node {
25f     Edge key;
c17     Node *l, *r;
981     ll delta;
a9c     void prop() {
6f9         key.w += delta;
d2d         if (l) l->delta += delta;
d86         if (r) r->delta += delta;
978         delta = 0;
0d3     }
866     Edge top() { prop(); return key; }
ab4 };
3eb Node *merge(Node *a, Node *b) {
b9f     if (!a || !b) return a ?: b;
626     a->prop(), b->prop();
dc2     if (a->key.w > b->key.w) swap(a, b);
485     swap(a->l, (a->r = merge(b, a->r)));
3f5     return a;
c51 }
7bb void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

408 pair<ll, vll> dmst(ll n, ll src, vector<Edge>& g) {
501     dsu_rb uf(n);
3f8     vector<Node*> heap(n);
563     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
cd2     ll res = 0;
b15     vll seen(n, -1), path(n), par(n);
169     seen[src] = src;
dd6     vector<Edge> Q(n), in(n, {-1,-1}), comp;
71b     deque<tuple<ll, ll, vector<Edge>>> cycs;
f5b     forn(s,0,n) {
02e         ll u = s, qi = 0, w;
a0a         while (seen[u] < 0) {
572             if (!heap[u]) return {-1,{}};
ebe             Edge e = heap[u]->top();
5ed             heap[u]->delta -= e.w, pop(heap[u]);
952             Q[qi] = e, path[qi++] = u, seen[u] = s;
d56             res += e.w, u = uf.find(e.a);
9e2             if (seen[u] == s) { /// found cycle, contract
28d                 Node* cyc = 0;
ab5                 ll end = qi, time = uf.time();
f38                 do cyc = merge(cyc, heap[w = path[--qi]]);
4ec                 while (uf.unite(u, w));
562                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
c06                 cycs.push_front({u, time, {&Q[qi], &Q[end]}});
417             }
26e         }
f23         forn(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
376     }

973     for (auto& [u,t,com] : cycs) { // restore sol (optional)
36c         uf.rollback(t);
1d0         Edge inEdge = in[u];
854         for (auto& e : com) in[uf.find(e.b)] = e;
56d         in[uf.find(inEdge.b)] = inEdge;
ac5     }
271     forn(i,0,n) par[i] = in[i].a;
efb     return {res, par};
7f3 }
```

2.9 Dominator Tree (Lengauer-Tarjan) 583

builds the dominator tree given a directed graph and a source. returns a vector where ret[u] is the immediate dominator of u.

Complexity: O((N+M) log N), O(N+M) mem

```
047 vll DomTree(vector<vll>& graph, ll src) {
9cf     ll n = sz(graph);
bab     vector<vll> tree(n), trans(n), buck(n);
2ea
    vll semi(n), par(n), dom(n), label(n), atob(n, -1), btoa(n, -1),
095         link(n, -1);

418     function<ll(ll, ll)> find=[&](ll u, ll d){
3d4         if(link[u] == -1) return d ? -1 : u;
9ff         ll v = find(link[u], d + 1);
388         if(v < 0) return u;
54b         if(semi[label[link[u]]] < semi[label[u]]) label[u] =
label[link[u]];
510         link[u] = v;
f95         return d ? v : label[u];
c83     };
b1b     ll t = 0;
5c2     function<void(ll)> dfs=[&](ll u){
640         atob[u] = t;
304         btoa[t] = u;
119         label[t] = semi[t] = t;
c82         t++;
612         for(auto v : graph[u]){
4f1         if(atob[v] == -1) dfs(v), par[atob[v]] = atob[u];
8b2             trans[atob[v]].push_back(atob[u]);
46c         }
155     };
088     dfs(src);
d48     for(ll u = t - 1; u >= 0; --u){
104         for(auto v : trans[u]) semi[u] = min(semi[u], semi[find(
v, 0)]);
9cf         if(u) buck[semi[u]].push_back(u);
9d5         for(auto w : buck[u]){
5d8             ll v = find(w, 0);
1db             dom[w] = semi[v] == semi[w] ? semi[w] : v;
c92         }
894         if(u) link[u] = par[u];
54f     }
ad9     vll ret(n, -1);
89d     forn(u,1,t) {
9c5         if(dom[u] != semi[u]) dom[u] = dom[dom[u]];
1eb         ret[btoa[u]] = btoa[dom[u]];
790     }
edf     return ret;
583 }
```

2.10 Euler Path / Euler Cycle 55d

Para declarar: 'euler<true> E(n);' se quiser direcionado e com 'n' vertices As funcoes retornam um par com um booleano indicando se possui o cycle/path que voce pediu, e um vector de {vertice, id da aresta para chegar no vertice} Se for get_path, na primeira posicao o id vai ser -1 get_path(src) tenta achar um caminho ou ciclo euleriano comecando no vertice 'src'. Se achar um ciclo, o primeiro e ultimo vertice serao 'src'. Se for um P3, um possiveo retorno seria [0, 1, 2, 0] get_cycle() acha um ciclo euleriano se o grafo for euleriano. Se for um P3, um possivel retorno seria [0, 1, 2] (vertie inicial nao repete) O(n+m)

```
63f template<bool directed=false> struct euler {
4de     ll n;
ad2     vector<vector<pll>> g;
eae     vll used;

e40     euler(ll n_) : n(n_), g(n) {}
176     void add(ll a, ll b) {
426         ll at = sz(used);
c51         used.push_back(0);
74e         g[a].emplace_back(b, at);
fab         if (!directed) g[b].emplace_back(a, at);
d86     }
d41 #warning chamar para o src certo!
dc4     pair<bool, vector<pll>> get_path(ll src) {
a4a         if (!sz(used)) return {true, {}};
528         vll beg(n, 0);
f1a         for (ll& i : used) i = 0;
         // {{vertice, anterior}, label}
ca8         vector<pair<pll, ll>> ret, st = {{{src, -1}, -1}};
df5         while (sz(st)) {
a9f             ll at = st.back().first.first;
da0             ll& it = beg[at];
dec
            while (it < sz(g[at]) && used[g[at][it].second]) it++;
902             if (it == sz(g[at])) {
0aa                 if (sz(ret) && ret.back().first.second != at)
b82                     return {false, {}};
420                 ret.push_back(st.back()), st.pop_back();
eb4             } else {
daa                 st.push_back({{g[at][it].first, at}, g[at][it].
```

```
second});
eb8                 used[g[at][it].second] = 1;
396             }
9d2         }
297         if (sz(ret) != sz(used)+1) return {false, {}};
307         vector<pll> ans;
fdf         for (auto i : ret) ans.emplace_back(i.first.first, i.
second);
459         reverse(ans.begin(), ans.end());
997         return {true, ans};
395     }
ae7     pair<bool, vector<pll>> get_cycle() {
a4a         if (!sz(used)) return {true, {}};
2af         ll src = 0;
eb5         while (!sz(g[src])) src++;
687         auto ans = get_path(src);
0e6         if (!ans.first || ans.second[0].first != ans.second.back
().first)
b82             return {false, {}};
350         ans.second[0].second = ans.second.back().second;
8b8         ans.second.pop_back();
ba7         return ans;
20c     }
55d };
```

2.11 Floyd-Warshall Algorithm

06d

Computes all-pairs shortest paths via DP over intermediate vertices. Initialize d[i][j] = edge weight (INF if no edge, 0 on diagonal). Returns true if a negative cycle is detected.

Complexity: O(N^3), O(N^2)

```
020 bool floyd_warshall(ll n, vector<vll>& d) {
b63     forn(k,0,n)
ded     forn(i,0,n)
b4c     forn(j,0,n)
db6         if (d[i][k] != INF && d[k][j] != INF)
0ab             d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

ded     forn(i,0,n)
7b3         if (d[i][i] < 0) return true;

d1f     return false;
06d }
```

2.12 Heavy Light Decomposition (Path query + Path update)

9b9

Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. All values initialized to the segtree default. Root must be 0. head[u]: Highest node (closest to root) of the heavy path containing u. pos[u]: Position of node u in the Segment Tree array (DFS order). Uses a Lazy Segment Tree

Complexity: O((log N)^2) per op, O(N) build

```
e8f #include "src/structures/segtree_lazy.cpp"

303 template <bool VALS_EDGES> struct HLD {
6fa     ll N, tim = 0;
059     vector<vll> adj;
65c     vll parent, siz, head, pos;
9a7     std::unique_ptr<segtree<Node, Update>> seg;
ffd     HLD(vector<vll> adj_)
d3e         : N(sz(adj_)), adj(adj_), parent(N, -1), siz(N, 1),
a3f           head(N),pos(N){ dfsSz(0); dfsHld(0);
daa             seg = make_unique<segtree<Node, Update>>(N);
0e4         }
987     void set_leaves(const vector<Node>& lvs) {
         // if VALS_EDGES lvs[i] is the weight of i -> parent[i]
384         vector<Node> leaves(N);
1a0         forn(i, 0, N) leaves[pos[i]] = lvs[i];
61b         seg->set_leaves(leaves);
a26     }
eb1     void dfsSz(ll v) { // get heavy son
d10         for (ll& u : adj[v]) {
44a             adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
308             parent[u] = v;
a00             dfsSz(u);
a02             siz[v] += siz[u];
68d             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
4af         }
774     }
7f7     void dfsHld(ll v) { // "linearizes" the tree
06c         pos[v] = tim++;
1c7         for (ll u : adj[v]) {
c0d             head[u] = (u == adj[v][0] ? head[v] : u);
9f9             dfsHld(u);
9af         }
e38     }
aa3     template <class B> void process(ll u, ll v, B op) {
2e7         for (;; v = parent[head[v]]) {
df0             if (pos[head[u]] > pos[head[v]]) swap(u, v);
a57             if (head[u] == head[v]) break;
ad9             op(pos[head[v]], pos[v]);
a34         }
bdc         if (pos[u] > pos[v]) swap(u, v);
adf         op(pos[u] + VALS_EDGES, pos[v]);
d5d     }
050     void modifyPath(ll u, ll v, Update upd) {
3d8         process(u, v, [&](ll l, ll r) {
576             seg->update(l, r, upd);
ebf         });
ce2     }
d0b     Node queryPath(ll u, ll v) {
0f1         Node res = Node();
         // Non-commutative ops require exact merge order.
3d8         process(u, v, [&](ll l, ll r) {
f23             res = res*seg->query(l, r);
9b9         });
b50         return res;
584     }
bb3     Node querySubtree(ll v) {
012
        return seg->query(pos[v] + VALS_EDGES, pos[v] + siz[v] - 1);
0f7     }
142     void modifySubtree(ll v, Update upd) {
e4c
        seg->update(pos[v] + VALS_EDGES, pos[v] + siz[v] - 1, upd);
aa8     }
6d1 };
```

2.13 Hungarian algorithm — min-cost perfect matching

e48

a[i][j] = cost of assigning worker i to job j (n workers, m >= n jobs) Returns {min cost, assignment}: assignment[i] = j

```
904 pair<ll, vll> hungarian(const vector<vll> &a) {
c04     if (a.empty()) return {0, {}};
1c6     ll n = sz(a) + 1, m = sz(a[0]) + 1;
544     vll u(n), v(m), p(m), ans(n - 1);
590     forn(i, 1, n) {
8c9         p[0] = i;
5f7         ll j0 = 0; // "dummy" worker 0
ead         vll dist(m, INF), pre(m, -1);
910         vector<bool> done(m + 1);
016         do { // dijkstra
781             done[j0] = true;
9cd             ll i0 = p[j0], j1 = 0, delta = INF;
6e0             forn(j, 1, m) if (!done[j]) {
f30                 auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
cf5                 if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
133                 if (dist[j] < delta) delta = dist[j], j1 = j;
a7c             }
b06             forn(j, 0, m) {
a5c                 if (done[j]) u[p[j]] += delta, v[j] -= delta;
202                 else dist[j] -= delta;
6da             }
6d4             j0 = j1;
f27         } while (p[j0]);
4b9         while (j0) { // update alternating path
b15             ll j1 = pre[j0];
0c1             p[j0] = p[j1], j0 = j1;
5b6         }
cf6     }
e73     forn(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
def     return {-v[0], ans}; // min cost
e48 }
```

2.14 LCA com Euler tour + Sparse Table (RMQ)

d01

Constroi com o grafo (lista de adjacencia) e a raiz 0 Assume arvore enraizada em 0 Complexidades: build - O(n log(n)) lca - O(1) dist - O(1)

```
4b6 #include "src/structures/sparse_table.cpp"

33e struct LCA {
3ec     ll T = 0;
475     vll time, path, ret, dep;
191     sparse rmq;

e21     LCA(vector<vll>& C, ll root) : time(sz(C)), dep(sz(C)), rmq
((dfs(C,root,-1), ret)) {}
5df     void dfs(vector<vll>& C, ll v, ll par) {
a0c         time[v] = T++;
06e         if(par != -1) dep[v] = dep[par] + 1;
24f         for (ll y : C[v]) if (y != par) {
cd1             path.push_back(v), ret.push_back(time[v]);
3d7             dfs(C, y, v);
c60         }
740     }

583     ll lca(ll a, ll b) {
6df         if (a == b) return a;
394         tie(a, b) = minmax(time[a], time[b]);
49d         return path[rmq.query(a, b-1)];
f06     }
949
    ll dist(ll a,ll b){return dep[a] + dep[b] - 2*dep[lca(a,b)];}
d90 };
```

2.15 LCA com binary lifting 6a7

Assume que um vertice eh ancestral dele mesmo, ou seja, se a eh ancestral de b, lca(a, b) = a MAXLOG = ceil(log(MAX)) Complexidades: build - O(n log(n)) lca - O(log(n))

```
ff2 vector<vll> g(MAX);
883 ll n, p;
a8e ll parent[MAXLOG][MAX];
d7e ll in[MAX], out[MAX];

65b void dfs(ll k) {
fdf     in[k] = p++;
2e0     for(ll v: g[k]){
e58         if(in[v] == -1){
5ad             parent[0][v] = k;
6b4             dfs(v);
e1f         }
ca0     }
26f     out[k] = p++;
65e }

f50 void build(ll raiz) {
4c7     forn(i,0,n) parent[0][i] = i;
c63     p = 0, memset(in, -1, sizeof in);
ecb     dfs(raiz);
671     forn(k,1,MAXLOG) forn(i,0,n)
ed8         parent[k][i] = parent[k - 1][parent[k - 1][i]];
7ce }

64c bool anc(ll a, ll b) { // se a eh ancestral de b
ef5     return in[a] <= in[b] && out[a] >= out[b];
bed }

583 ll lca(ll a, ll b) {
86d     if (anc(a, b)) return a;
e52     if (anc(b, a)) return b;
fb3     for (ll k = MAXLOG - 1; k >= 0; k--)
499         if (!anc(parent[k][a], b)) a = parent[k][a];
793     return parent[0][a];
2ab }
```

2.16 Min Cost Max Flow (MCMF) f76

min_cost_flow(s, t, f) computes the pair (flow, cost) where max(flow) <= f with min(cost). min_cost_flow(s, t) -> Maximum flow with minimum cost from s to t. For DAGs, SPFA can be replaced with DP to avoid O(nm) initial cost. If no negative cost edges exist, SPFA is not needed.

Complexity: O(nm + f * m log n), O(n + m)

```
123 template<typename T> struct mcmf {
670     struct edge {
435         ll to, rev, flow, cap; // destination, reverse edge id, current flow, capacity
7f9         bool res; // is reverse edge
635         T cost; // cost per unit of flow
892         edge() : to(0), rev(0), flow(0), cap(0), cost(0), res(false) {}
92e
        edge(ll to_, ll rev_, ll flow_, ll cap_, T cost_, bool res_)
f8d             : to(to_), rev(rev_), flow(flow_), cap(cap_), res(res_), cost(cost_) {}
2cb     };

002     vector<vector<edge>> g;
e04     vll par_idx, par;
f1e     T inf;
a03     vector<T> dist;

673     mcmf(ll n) : g(n), par_idx(n), par(n), inf(numeric_limits<T>::max()/3) {}

6dd     void add(ll u, ll v, ll w, T cost) { // edge from u to v with capacity w and cost
efb         edge a = edge(v, sz(g[v]), 0, w, cost, false);
32a         edge b = edge(u, sz(g[u]), 0, 0, -cost, true);

b24         g[u].push_back(a);
c12         g[v].push_back(b);
fc8     }

b27
    vector<T> spfa(ll s) { // not needed if no negative cost edges
ed2         deque<ll> q;
9f3         vector<bool> is_inside(sz(g), 0);
d36         dist = vector<T>(sz(g), inf);

a93         dist[s] = 0;
a30         q.push_back(s);
ecb         is_inside[s] = true;

14d         while (!q.empty()) {
017             ll v = q.front();
ced             q.pop_front();
48d             is_inside[v] = false;

2ec             forn(i,0,sz(g[v])){
9d4                 auto [to, rev, flow, cap, res, cost] = g[v][i];
29e                 if (flow < cap && dist[v] + cost < dist[to]) {
943                     dist[to] = dist[v] + cost;

ed6                     if (is_inside[to]) continue;
506                     if (!q.empty() && dist[to] > dist[q.front()]) q.push_back(to);
b33                     else q.push_front(to);
b52                     is_inside[to] = true;
fa8                 }
fce             }
027         }
8d7         return dist;
603     }
9aa     bool dijkstra(ll s, ll t, vector<T>& pot) {
90e         priority_queue<pair<T, ll>, vector<pair<T, ll>>, greater<>> q;
d36         dist = vector<T>(sz(g), inf);
a93         dist[s] = 0;
115         q.emplace(0, s);
647         while (sz(q)) {
91b             auto [d, v] = q.top();
833             q.pop();
68b             if (dist[v] < d) continue;
2ec             forn(i,0,sz(g[v])) {
9d4                 auto [to, rev, flow, cap, res, cost] = g[v][i];
e8c                 cost += pot[v] - pot[to];
29e                 if (flow < cap && dist[v] + cost < dist[to]) {
943                     dist[to] = dist[v] + cost;
441                     q.emplace(dist[to], to);
88b                     par_idx[to] = i, par[to] = v;
96e                 }
347             }
38f         }
1d4         return dist[t] < inf;
b24     }

928     pair<ll, T> min_cost_flow(ll s, ll t, ll flow = INF) {
437         vector<T> pot(sz(g), 0);
9e4         pot = spfa(s); // change shortest path algorithm here
            // no negative costs: nothing needed
            // DAG: use DP instead
c62         ll f = 0;
ce8         T ret = 0;
472         while (f < flow && dijkstra(s, t, pot)) {
945             forn(i,0,sz(g))
d2a                 if (dist[i] < inf) pot[i] += dist[i];

306             ll mn_flow = flow - f, u = t;
045             while (u != s){
90f                 mn_flow = min(mn_flow,
07d                     g[par[u]][par_idx[u]].cap - g[par[u]][par_idx[u]].flow);
3d1                 u = par[u];
935             }

1f2             ret += pot[t] * mn_flow;

476             u = t;
045             while (u != s) {
e09                 g[par[u]][par_idx[u]].flow += mn_flow;
d98                 g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
3d1                 u = par[u];
bcc             }

04d             f += mn_flow;
2b4         }

15b         return make_pair(f, ret);
2e1     }

    // Optional: returns original edges where flow == cap
1ee     vector<pll> recover() {
fcb         vector<pll> used;
b25         forn(i, 0, sz(g)) for (edge e : g[i])
587
            if(e.flow == e.cap && !e.res) used.push_back({i, e.to});
f6b         return used;
580     }
f76 };
```

2.17 Strongly Connected Components (Kosaraju) fdf

Computes SCCs using two DFS passes and builds the condensation graph.

Complexity: O(N + M), O(N + M)

```
591 vector<bool> visited;

775 void dfs(ll v, vector<vll>& g, vll &out) {
e75     visited[v] = true;
819     for(auto u : g[v]) if(!visited[u]) dfs(u, g, out);
3ad     out.push_back(v);
043 }

b19 vector<vll> scc(vector<vll>& g) {
```

```
625     int n = sz(g);
629     vll order, roots(n, 0);

4cc     vector<vll> adj_rev(n);
0c2     forn(u, 0, n) for (ll v : g[u]) adj_rev[v].push_back(u);

411     visited.assign(n, false);
2b4     forn(i, 0, n) if (!visited[i]) dfs(i, g, order);
b3a     reverse(order.begin(), order.end());

411     visited.assign(n, false);
de0     ll curr_comp = 0;
0ee     for (auto v : order) {
451         if (!visited[v]) {
291             vll component; dfs(v, adj_rev, component);
fe3             for (auto u : component) roots[u] = curr_comp;
5f2             curr_comp++;
e3f         }
d55     }

e94     set<pll> edges;
361     vector<vll> cond_g(curr_comp);
c6b     forn(u, 0, n) {
7b9         for (auto v : g[u]) {
2dd             if (roots[u] != roots[v] && !edges.count({roots[u],
roots[v]})) {
2b9                 cond_g[roots[u]].push_back(roots[v]);
893                 edges.emplace(roots[u], roots[v]);
fbe             }
f76         }
3b9     }
594     return cond_g;
c8d }
```

2.18 Topological Sort (Kahn's Algorithm) 4fe

Produces a topological ordering of a DAG using indegree counting and a queue-like frontier.

Complexity: O(N + M), O(N)

```
af3 vll topo_sort(const vector<vll>& g) {
a93     vll indeg(sz(g)), q;
edb     for (auto& li : g) for (int x : li) indeg[x]++;
707     forn(i, 0, sz(g)) if (indeg[i] == 0) q.push_back(i);
7c5     forn(j, 0, sz(q)) for(int x : g[q[j]]) if(--indeg[x] == 0) q.
push_back(x);
bef     return q;
4fe }
```

2.19 Virtual Tree b8d

*Comprime uma arvore dado um conjunto S de vertices, de forma que o conjunto de vertices da arvore comprimida contenha S e seja minimal e fechado sobre a operacao de LCA Se |S| = k, a arvore comprimida tem menos que 2k vertices As arestas de virt possuem a distancia do vertice ate o vizinho em arestas Retorna a raiz da virtual tree Precisa: lca::pos deve ser a ordem de visitacao no dfs lca::lca, lca::dist O(k * T), T tempo do lca::lca*

```
aa4 #include "src/graphs/lca.cpp"

1fd const ll MAX = 100'005;
00e vector<pll> virt[MAX];

b8d ll build_virt(vll v, LCA& lca) {
06b     auto cmp = [&](ll i, ll j) { return lca.time[i] < lca.time[j
]; };
074     sort(v.begin(), v.end(), cmp);
f4d
    for (ll i = sz(v)-1; i; i--) v.push_back(lca.lca(v[i], v[i-1]));
074     sort(v.begin(), v.end(), cmp);
d76     v.erase(unique(v.begin(), v.end()), v.end());
9e2     forn(i,0,sz(v)) virt[v[i]].clear();
86c     forn(i,1,sz(v)) virt[lca.lca(v[i-1], v[i])].clear();
c03     forn(i,1,sz(v)) {
d33         ll parent = lca.lca(v[i-1], v[i]);
b82         ll d = lca.dist(parent, v[i]);
 // #warning soh to colocando aresta descendo
4d0         virt[parent].emplace_back(v[i], d);
1e4     }

832     return v[0];
c47 }
```

3 Mathematics

3.1 Chinese Remainder Theorem 603

se x = a(mod m) e x = b(mod n) então existe unico c<mmc(m,n) tal que x = c(mod mmc(m,n)) se e só se mdc(m,n)| (b-a) essa funcao acha c e é -INF se for impossivel

```
b12 #include "src/math/euclid.cpp"

24a ll crt(ll a, ll m, ll b, ll n) {
08f     if (n > m) swap(a, b), swap(m, n);
58f     ll x, y, g = euclid(m, n, x, y);
51a     if((a - b) % g != 0) return -INF; //no solution
ef8     x = (b - a) % n * x % n / g * m + a;
16c     return x < 0 ? x + m*n/g : x;
80d }
```

3.2 Extended Eucidian Algorithm 33b

Acha mdc(a,b) e x e y que satisfazem a identidade de bezout

```
c22 ll euclid(ll a, ll b, ll &x, ll &y) {
1ee     if (!b) return x = 1, y = 0, a;
e3d     ll d = euclid(b, a % b, y, x);
0a4     return y -= a/b * x, d;
33b }
```

3.3 FFT/NTT Convolution c3c

Implements iterative FFT over complex numbers and NTT over supported primes; provides convolution utility.

Complexity: O(N log N), O(N)

```
360 #include "src/math/mint.cpp" // for nnt

// Para FFT
1f1 void get_roots(bool f, ll n, vector<complex<double>>& roots) {
f26     const static double PI = acosl(-1);
790     forn(i,0,n/2){
b1e         double alpha = i*((2*PI)/n);
1a1         if (f) alpha = -alpha;
069         roots[i] = {cos(alpha), sin(alpha)};
d54     }
855 }

// Para NTT
6c0 template<ll p>
206 void get_roots(bool f, ll n, vector<mod_int<p>>& roots) {
1e6     mod_int<p> r;
bfe     ll ord;
57a     if(p == 998244353){
9b6         r = 102292;
81b         ord = (1 << 23);
121     } else if (p == 754974721) {
43a         r = 739831874;
f0a         ord = (1 << 24);
d48     } else if (p == 167772161) {
a2a         r = 243;
033         ord = (1 << 25);
5a4     } else assert(false);

547     if (f) r = r^(p - 1 -ord/n);
ee2     else r = r^(ord/n);
be4     roots[0] = 1;
098     forn(i,1,n/2) roots[i] = roots[i-1]*r;
663 }

48e template<typename T> void fft(vector<T>& a, bool f, ll N, vector<
ll>& rev) {
73b     forn(i,0,N) if (i < rev[i]) swap(a[i], a[rev[i]]);
816     ll l, r, m;
cb4     vector<T> roots(N);
0ec     for (ll n = 2; n <= N; n *= 2) {
0f4         get_roots(f, n, roots);
dc1         for (ll pos = 0; pos < N; pos += n) {
432             l = pos + 0, r = pos + n/2, m = 0;
a88             while (m < n/2) {
297                 auto t = roots[m] * a[r];
254                 a[r] = a[l] - t;
b8f                 a[l] = a[l] + t;
2c9                 l++, r++, m++;
d89             }
f6c         }
0c1     }
235     if (f) {
1c5         auto invN = T(1) / T(N);
37b         for (ll i = 0; i < N; i++) a[i] = a[i] * invN;
45d     }
78d }

bf5 template<typename T> vector<T> convolution(vector<T>& a, vector<
T>& b) {
87a     vector<T> l(a.begin(), a.end()), r(b.begin(), b.end());
3f1     ll N = sz(l)+sz(r)-1;
a10     ll n = 1, log_n = 0;
8cf     while (n < N) n *= 2, log_n++;
e66     vector<ll> rev(n);
522     forn(i,0,n) {
434         rev[i] = 0;
0d7         forn(j,0,log_n) if (i>>j&1)
4ff             rev[i] |= 1 << (log_n-1-j);
ac6     }

143     assert(N <= n);
fa4     l.resize(n);
7e4     r.resize(n);
56e     fft(l, false, n, rev);
fcf     fft(r, false, n, rev);
a46     for (ll i = 0; i < n; i++) l[i] *= r[i];
88b     fft(l, true, n, rev);
```

```
5e1     l.resize(N);
792     return l;
e83 }

// NTT
4a7 template<ll p, typename T> // to cast vll to vector<mint>
b74 vector<mod_int<p>> ntt(vector<T>& a, vector<T>& b) {
d52
    vector<mod_int<p>> A(a.begin(), a.end()), B(b.begin(), b.end());
d29     return convolution(A, B);
543 }
```

3.4 Divisor Decomposition / Square Root Trick `c63`

Itera sobre intervalos [l, r] onde floor(n/i) é constante Complexidade: O(sqrt(n))

```
0aa void divisor_decomposition(ll n) {
d13     for (ll l = 1, r; l <= n; l = r + 1) {
025         ll val = n / l;
740         r = n / val;
         // n/i == val para todo i em [l, r]
         // Exemplo: res += (r - l + 1) * val;
566     }
c63 }
```

3.5 Gauss `def`

*Resolve sistema linear Retornar um par com o numero de solucoes e alguma solucao, caso exista O(n^2 * m) escrito pelo gpt, cuidado*

```
67a template<typename T>
915 pair<ll, vector<T>> gauss(vector<vector<T>> a, vector<T> b) {
6ca     const double eps = 1e-6;
ee9     ll n = sz(a), m = sz(a[0]);
f4c     forn(i, 0, n) a[i].push_back(b[i]);

e47     vll where(m, -1);
d41     for (ll col = 0, row = 0; col < m && row < n; col++) {
b49         ll sel = row;
d96         forn(i, row, n)
e55             if(abs(a[i][col]) > abs(a[sel][col])) sel = i;
2c4         if(abs(a[sel][col]) < eps) continue;
552         forn(i, col, m+1)
dd2             swap(a[sel][i], a[row][i]);
2c3         where[col] = row;

959         forn(i, 0, n) if (i != row) {
96c             T c = a[i][col] / a[row][col];
b2d             forn(j, col, m+1)
c8f                 a[i][j] -= a[row][j] * c;
5a4         }
b70         row++;
11c     }

b1d     vector<T> ans(m, 0);
363     forn(i, 0, m) if (where[i] != -1)
12a         ans[i] = a[where[i]][m]/a[where[i]][i];
522     forn(i, 0, n) {
501         T sum = 0;
695         forn(j, 0, m)
5a9             sum += ans[j]*a[i][j];
b1f         if(abs(sum - a[i][m]) > eps)
6cd             return pair(0, vector<T>());
417     }
688     forn(i, 0, m) if (where[i] == -1)
018         return pair(INF, ans);
280     return pair(1, ans);
965 }
```

3.6 Gauss - Z2 `5e6`

D eh dimensao do espaco vetorial add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base) coord(v) - retorna as coordenadas (c) de v na base atual (basis[T].c == v) recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos coord(v).first e recover(v).first - se v pertence ao span Complexidade: add, coord, recover: O(D[2 / 64)

```
502 template<ll D> struct gauss_z2 {
3c1     bitset<D> basis[D], keep[D];
1cf     ll rk, in, dim;
d6f     vll id;

cf2     gauss_z2 () : rk(0), in(-1), id(D, -1) {};

61e     gauss_z2 (ll dim_) : rk(0), in(-1), id(D, -1), dim(dim_) {};

04e     bool add(bitset<D> v) {
42c         in++;
fb0         bitset<D> k;
294         for (ll i = dim - 1; i >= 0; i--) if (v[i]) {
189             if (basis[i][i]) v ^= basis[i], k ^= keep[i];
4e6             else {
ea6                 k[i] = true, id[i] = in, keep[i] = k;
6ce                 basis[i] = v, rk++;
8a6                 return true;
b34             }
347         }
d1f         return false;
60f     }
0f6     pair<bool, bitset<D>> coord(bitset<D> v) { // private
944         bitset<D> c;
294         for (ll i = dim - 1; i >= 0; i--) if (v[i]) {
a39             if (basis[i][i]) v ^= basis[i], c[i] = true;
8af             else return {false, bitset<D>()};
7bd         }
5db         return {true, c};
85e     }
761     pair<bool, vll> recover(bitset<D> v) {
22e         auto [span, bc] = coord(v);
af8         if (not span) return {false, {}};
f79         bitset<D> aux;
e24
            for (ll i = dim - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
79a         vll oc;
369         for (ll i = dim - 1; i >= 0; i--) if (aux[i]) oc.
push_back(id[i]);
001         return {true, oc};
e71     }
5e6 };
```

3.7 Flattened Divisor Sieve (CSR format) `022`

Precomputes all divisors up to n. Faster than vector<vll> for cache reasons Time Complexity: O(n log n) | Space Complexity: O(n log n)

```
45c tuple<vll, vll, vll> get_divisor(ll n) {
70b     vll head(n);
651     vll tail(n);
cbe     vll divs(n*(log(n)+1));
d44     forn(i, 1, n)
c6c         for (ll j = i; j < n; j += i)
987             head[j]++;

870     ll curr = 0;
590     forn(i, 1, n) {
825         ll cnt = head[i];
8a9         head[i] = curr;
6f2         tail[i] = curr;
c71         curr += cnt;
d6b     }

d44     forn(i, 1, n)
c6c         for (ll j = i; j < n; j += i)
1b6             divs[tail[j]++] = i;
66f     return {head, tail, divs};
022 }
// for d | x =
 // forn(k,head[x], tail[x]){
 //     ll d = divs[k];
// }
```

3.8 Matrix template `caa`

- operator: multiplicacao (O(n*m*r)) opcional modo modular (#define MODULAR true) - operator[]: exponenciacao rapida (O(log e)) - apply_transform(): aplica transformacao M[e * v*

```
945 #define MODULAR false
5ed template<typename T> struct matrix : vector<vector<T>> {
03e     ll n, m;
30f     void print() {
522         forn(i,0,n) {
064             forn(j,0,m) cout << (*this)[i][j] << " ";
1fb             cout << endl;
285         }
823     }

491     matrix(ll n_, ll m_, bool ident = false) :
b14
            vector<vector<T>>(n_, vector<T>(m_, 0)), n(n_), m(m_) {
94e         if (ident) {
df7             assert(n == m);
17a             forn(i,0,n) (*this)[i][i] = 1;
636         }
f73     }
b83     matrix(const vector<vector<T>>& c) : vector<vector<T>>(c),
98e         n(sz(c)), m(sz(c[0])) {}
efc     matrix(const initializer_list<initializer_list<T>>& c) {
f7e         vector<vector<T>> val;
212         for (auto& i : c) val.push_back(i);
303         *this = matrix(val);
c50     }

388     matrix<T> operator*(matrix<T>& r) {
1e2         assert(m == r.n);
82c         matrix<T> M(n, r.m);
630         forn(i,0,n) forn(k,0,m) forn(j, 0, r.m){
e34             T add = (*this)[i][k] * r[k][j];
f98 #if MODULAR
d41 #warning Usar matrix<ll> e soh colocar valores em [0, MOD) na
matriz!
8b6             M[i][j] += add%MOD;
```

```
983             if (M[i][j] >= MOD) M[i][j] -= MOD;
8c1 #else
7bb             M[i][j] += add;
f2e #endif
77d         }
474         return M;
ba1     }
528     matrix<T> operator^(ll e){
f10         matrix<T> M(n, n, true), at = *this;
c87         while (e) {
2e2             if (e&1) M = M*at;
cc2             e >>= 1;
c80             at = at*at;
eb6         }
474         return M;
ca3     }
582     void apply_transform(matrix M, ll e){
1c3         auto& v = *this;
c87         while (e) {
9ba             if (e&1) v = M*v;
cc2             e >>= 1;
419             M = M*M;
d86         }
4e5     }
f02 };
```

3.9 Miller-Rabin Primality Test (Deterministic for 64-bit) bdb

Checks if a number N is prime using a fixed set of bases that guarantees correctness for any N < 2^64. Uses a long double trick for O(1) modular multiplication.

Complexity: O(7 log^3 N) per query

```
924 #include "src/math/mod_functions.cpp"

da4 bool isPrime(ull n){
c16     if(n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
ef3     ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
 s = __builtin_ctzll(n-1), d = n >> s;
e80     for (ull a : A) {
6b4         ull p = modpow(a%n, d, n), i = s;
274         while(p != 1 && p != n-1 && a % n && i--)
c77             p = modmul(p, p, n);
e28         if(p != n-1 && i != s) return 0;
edf     }
6a5     return 1;
60d }
```

3.10 Modular Integer 6eb

*Fixed-modulus integer type with +, -, *, /, and exponentiation; modulo should be prime for division via Fermat.*

Complexity: O(1) per arithmetic op (O(log E) for exponentiation), O(1)

```
a2f const ll MOD = 998244353;

f25 template<ll p> struct mod_int {
c68     ll expo(ll b, ll e) {
c85         ll ret = 1;
c87         while (e) {
cad             if (e % 2) ret = ret * b % p;
9d2             e /= 2, b = b * b % p;
c42         }
edf         return ret;
734     }
1f6     ll inv(ll b) { return expo(b, p-2); }
4d7     using m = mod_int;
aa3     ll v;
fe0     mod_int() : v(0) {}
e12     mod_int(ll v_) {
c8e         if (v_ >= p || v_ <= -p) v_ %= p;
bc6         if (v_ < 0) v_ += p;
2e7         v = v_;
5d7     }
74d     m& operator +=(const m& a) {
2fd         v += a.v;
ba5         if (v >= p) v -= p;
357         return *this;
c8b     }
eff     m& operator -=(const m& a) {
8b4         v -= a.v;
cc8         if (v < 0) v += p;
357         return *this;
f8d     }
4c4     m& operator *=(const m& a) {
686         v = v * a.v % p;
357         return *this;
c9b     }
3f9     m& operator /=(const m& a) {
5d6         v = v * inv(a.v) % p;
357         return *this;
62d     }
f98     m operator -() const { return m(-v); }
b3e     m& operator ^=(ll e) {
06d         if (e < 0) {
6e2             v = inv(v);
00c             e = -e;
275         }
284         v = expo(v, e);
        // possible optimization:
        // careful with 0^0
        // v = expo(v, e%(p-1));
357         return *this;
6ed     }
423     bool operator ==(const m& a) { return v == a.v; }
69f     bool operator !=(const m& a) { return v != a.v; }

1c6     friend istream& operator >>(istream& in, m& a) {
d1c         ll val; in >> val;
d48         a = m(val);
091         return in;
870     }
44f     friend ostream& operator <<(ostream& out, m a) {
5a0         return out << a.v;
214     }
399     friend m operator +(m a, m b) { return a += b; }
f9e     friend m operator -(m a, m b) { return a -= b; }
9c1     friend m operator *(m a, m b) { return a *= b; }
51b     friend m operator /(m a, m b) { return a /= b; }
08f     friend m operator ^(m a, ll e) { return a ^= e; }
9ad };
0f4 typedef mod_int<MOD> mint;
```

3.11 Modular Arithmetic Helpers a36

Provides fast modular mul, fast exponentiation.

Complexity: O(log E) for power/inverse

```
f4c typedef unsigned long long ull;

b4d const ll MOD = 1'000'000'007;

f85 ull modmul(ull a, ull b, ull M){
2dd     ll ret = a*b - M * ull(1.L / M*a*b);
964     return ret + M * (ret < 0) - M * (ret >= (ll)M);
68d };

4f6 ull modpow(ull b, ull e, ull mod){
c1a     ull ans = 1;
a18     for(; e; b = modmul(b, b, mod), e /= 2)
9e8         if(e & 1) ans = modmul(ans, b, mod);
ba7     return ans;
100 }
```

3.12 Discrete Logarithm (Baby-step Giant-step) b26

Finds the smallest x >= 1 such that a^x = b (mod m), or returns -1 if no solution exists. Uses Baby-step Giant-step with a hash map for O(sqrt(m)) lookups.

Complexity: O(sqrt(m) * log(sqrt(m)))

```
a4f ll mod_log(ll a, ll b, ll m) {
462     ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
e2d     map<ll, ll> mp;
8b3     while (j <= n && (e = f = e * a % m) != b % m)
46e         mp[e * b % m] = j++;
671     if (e == b % m) return j;
beb     if (gcd(m, e) == gcd(m, b))
2c2         forn(i,2,n+2) if (mp.count(e = e * f % m))
bce             return n * i - mp[e];
daa     return -1;
b26 }
```

3.13 Discrete Square Root ef7

Tonelli-Shanks para encontrar x tq x^2 = a (mod p) O(sqrt(p))

```
924 #include "src/math/mod_functions.cpp"

a77 ll sqrt(ll a, ll p) {
5de     a %= p; if (a < 0) a += p;
b47     if (a == 0) return 0;
709     if(modpow(a, (p-1)/2, p) != 1) return -1; // se não é
possivel fala -1
a75     if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
b94     ll s = p - 1, n = 2;
ee5     int r = 0, m;
084     while (s % 2 == 0)
082         ++r, s /= 2;
    /// find a non-square mod p
eaa     while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
0c3     ll x = modpow(a, (s + 1) / 2, p);
b74     ll b = modpow(a, s, p), g = modpow(n, s, p);
1af     for (;; r = m) {
4fd         ll t = b;
713         for (m = 0; m < r && t != 1; ++m)
c58             t = t * t % p;
ae0         if (m == 0) return x;
20e         ll gs = modpow(g, 1LL << (r - m - 1), p);
fba         g = gs * gs % p;
4fb         x = x * gs % p;
c5c         b = b * g % p;
e3a     }
b7c }
```

3.14 Primitive Root (Generator) 4ce

Finds the smallest primitive root modulo a prime P. Factorizes P-1 and checks that for every prime factor F of P-1, the candidate G satisfies G^((P-1)/F) != 1 (mod P). can be adapted to p not prime

Complexity: O(time_to_factor(p-1) + ans * |factors(P-1)| * log P)

```
924 #include "src/math/mod_functions.cpp"

e7f ll primitive_root(ll p){
1e7     vll fact;
546     ll phi = p-1,  n = phi;
    // mudar p-1 para caso nao primo
b6d     for (ll i=2; i*i<=n; i++)
c06         if (n % i == 0) {
a54             fact.push_back (i);
49e             while (n % i == 0)
135                 n /= i;
7a0         }
f3d     if (n > 1)
d24         fact.push_back(n);

293     forn(res,2, p+1) {
971         bool ok = true;
557         for (ll i=0; i<sz(fact) && ok; ++i)
754             ok &= modpow (res, phi / fact[i], p) != 1;
97c         if (ok)  return res;
504     }
daa     return -1;
23e }
```

3.15 Quadratic Field Extension (Z_p[sqrt(D)]) b77

*Extension for numbers of form x + y*sqrt(D) with +, -, *, /, and exponentiation.*

Complexity: O(1) per arithmetic op, O(log MOD) for division/inversion.

```
360 #include "src/math/mint.cpp"

41a template <long long D>
281 struct quad {
14f     mint x, y;
f0c     using m = quad<D>;

c72     quad() : x(0), y(0) {}
565     quad(ll x_) : x(x_), y(0) {}
9e9     quad(mint x_, mint y_) : x(x_), y(y_) {}

74d     m& operator += (const m& a) {
12b         x += a.x; y += a.y; return *this;
30f     }
eff     m& operator -= (const m& a) {
dd8         x -= a.x; y -= a.y; return *this;
a07     }

4c4     m& operator *= (const m& a) {
86a         mint old_x = x;
1df         x = x * a.x + mint(D) * y * a.y;
275         y = old_x * a.y + y * a.x;
357         return *this;
099     }

ad4     m inv() const {
04b         mint dem = x * x - mint(D) * y * y;
117         return m(x / dem, -y / dem);
2b5     }
3f9     m& operator /=(const m& a) {
8f8         *this *= a.inv();
357         return *this;
201     }

8de     m expo(m b, ll e){
7f9         m ret = m(1, 0);
c87         while (e) {
9a9             if (e % 2) ret = ret * b;
e04             b = b * b;
e1c             e /= 2;
9a5         }
edf         return ret;
bc1     }

b3e     m& operator ^=(ll e) {
06d         if (e < 0) {
625             *this = this->inv();
00c             e = -e;
019         }
670         *this = expo(*this, e);
357         return *this;
7b9     }

1c2     bool operator ==(const m& a){ return (x == a.x) && (y == a.y); }
86b     bool operator !=(const m& a){ return (x != a.x) || (y != a.y); }

b12     friend ostream& operator <<(ostream& out, const m& a) {
7fb         return out << a.x << ":" << a.y;
1b1     }

5a6     friend m operator +(m a, const m& b) { return a += b; }
022     friend m operator -(m a, const m& b) { return a -= b; }
cfb     friend m operator *(m a, const m& b) { return a *= b; }
a1e     friend m operator /(m a, const m& b) { return a /= b; }
23d     friend m operator ^(m a, ll b) { return a ^= b; }
298 };
```

3.16 Linear Sieve (prime precomputation) 70c

Computes primes up to N in O(N) using the smallest prime factor (lp). pr will contain all primes <= N. lp[x] stores the smallest prime dividing x.

Complexity: O(N)

```
936 pair<vll, vll> sieve(ll n){
309     vll lp(n+1);
e67     vll pr;
3e5     forn(i, 2, n+1) {
719         if (lp[i] == 0) {
116             lp[i] = i;
bc0             pr.push_back(i);
d4f         }
778         for (ll j = 0; i * pr[j] <= n; ++j) {
a97             lp[i * pr[j]] = pr[j];
5f9             if (pr[j] == lp[i]) {
c2b                 break;
e08             }
9b9         }
8d6     }
52d     return {pr, lp};
728 }
```

```
638 vll calc_mu(vll& lp) {
c96     ll n = sz(lp) - 1;
caf     vll ans(n);
ca9     ans[1] = 1;
aff     forn(i,2,n) {
5ea         ll p = lp[i], x = i/p;
08c         if (lp[x] == p) ans[i] = 0;
bcf         else ans[i] = -ans[x];
29f     }
ba7     return ans;
bc2 }

823 vll calc_d(vll& lp) {
c96     ll n = sz(lp) - 1;
caf     vll ans(n);
ca9     ans[1] = 1;
aff     forn(i, 2, n) {
5ea         ll p = lp[i], x = i / p;
f85         if (lp[x] != p) ans[i] = 1;
4f9         else ans[i] = ans[x] + 1;
cc9     }
aff     forn(i, 2, n) {
5ea         ll p = lp[i], x = i / p;
055         if (ans[i] == 1) ans[i] = ans[x] * 2;
b3f         else ans[i] = (ans[x] / ans[i]) * (ans[i] + 1);
bff     }
ba7     return ans;
803 }
```

4 String Algorithms

4.1 Aho-Corasick Automaton 825

Builds a trie with failure links for multi-pattern matching; insert is O(|s|), build is linear in total length, and queries run in linear time in the text.

Complexity: varies, O(total patterns length)

```
ea1 namespace aho {
05b     map<char, ll> to[MAX];
b0a     ll link[MAX], idx, term[MAX], exit[MAX], sobe[MAX];
663     vll max_match(MAX, 0);

bfc     void insert(string& s) {
4eb         ll at = 0;
b4f         for (char c : s) {
b68             auto it = to[at].find(c);
1c9             if (it == to[at].end()) at = to[at][c] = ++idx;
361             else at = it->second;
ff4         }
142         term[at]++, sobe[at]++;
6b7         max_match[at] = sz(s);
5f4     }

0a8     void build() {
848         queue<ll> q;
537         q.push(0);
dff         link[0] = exit[0] = -1;
647         while (sz(q)) {
aa7             ll i = q.front(); q.pop();
3c4             for (auto [c, j] : to[i]) {
7d8                 ll l = link[i];
34a                 while (l != -1 && !to[l].count(c)) l = link[l];
```

```
7a5                 link[j] = l == -1 ? 0 : to[l][c];
3ab
            exit[j] = term[link[j]] ? link[j] : exit[link[j]];

058                 max_match[j] = max(max_match[link[j]], max_match
[j]);
6f2                 if (exit[j]+1) sobe[j] += sobe[exit[j]];
113                 q.push(j);
c8a             }
3ce         }
a63     }
5e1     ll query(string& s) {
0db         ll at = 0, ans = 0;
b4f         for (char c : s){
475             while (at != -1 && !to[at].count(c)) at = link[at];
5b9             at = at == -1 ? 0 : to[at][c];
2b1             ans += sobe[at];
1e7         }
ba7         return ans;
7df     }
06d     vll match_vec(string& s) {
c83         ll at = 0, n = sz(s);
d26         vll v(n, 0);
522         forn(i, 0, n){
827             char c = s[i];
475             while (at != -1 && !to[at].count(c)) at = link[at];
5b9             at = at == -1 ? 0 : to[at][c];

c84             v[i] = max_match[at]; // quero isso
2f2         }
6dc         return v;
2ff     }
825 }
```

4.2 Knuth-Morris-Pratt (KMP) 0cf

Computes prefix function and performs linear-time substring search with optional automaton construction.

Complexity: O(n + m), O(n)

```
b9d vll pi(string& s) {
36f     vll p(sz(s));
3b9     for (ll i = 1, j = 0; i < sz(s); i++) {
5c2         while (j && s[j] != s[i]) j = p[j-1];
973         if (s[j] == s[i]) j++;
f8c         p[i] = j;
00d     }
74e     return p;
bac }

ce5 vll match(string& pat, string& s) {
2b9     vll p = pi(pat), match;
c0f     for (ll i = 0, j = 0; i < sz(s); i++) {
6e6         while (j && pat[j] != s[i]) j = p[j-1];
3e3         if (pat[j] == s[i]) j++;
1f4         if (j == sz(pat)) match.push_back(i-j+1), j = p[j-1];
ae5     }
ed8     return match;
07e }

2c7 struct KMPaut : vector<vll> {
47c     KMPaut(){}
bb4     KMPaut (string& s) : vector<vll>(26, vll(sz(s)+1)) {
411         vll p = pi(s);
04b         auto& aut = *this;
4fa         aut[s[0]-'a'][0] = 1;
19a         for (char c = 0; c < 26; c++)
906             for (int i = 1; i <= sz(s); i++)
42b
                aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
811     }
b86 };
```

4.3 Suffix Array - O(n log n) f4b

kasai recebe o suffix array e calcula lcp[i], o lcp entre s[sa[i],...,n-1] e s[sa[i+1],..,n-1] Complexidades: suffix_array - O(n log(n)) kasai - O(n)

```
9f3 vll suffix_array(string s) {
59b     s.push_back('$'); // 0 caso vll (CHECAR SE PODE)
14a     ll n = sz(s), N = max(n, 260ll);
eaf     vll sa(n), ra(n);
828     forn(i, 0, n) sa[i] = i, ra[i] = s[i];

2e6     for(ll k = 0; k < n; k ? k *= 2 : k++) {
dad         vll nsa(sa), nra(n), cnt(N);

fae         for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[
ra[i]]++;

e18         forn(i, 1, N) cnt[i] += cnt[i-1];
f62
        for(ll i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

aec         for(ll i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa
[i]] !=
c45             ra[sa[i-1]] || ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
26b         ra = nra;
d5e         if (ra[sa[n-1]] == n-1) break;
b39     }
649     return vll(sa.begin()+1, sa.end());
3c4 }

586 vll kasai(string s, vll sa) {
c30     ll n = sz(s), k = 0;
1b5     vll ra(n), lcp(n);
540     forn(i, 0, n) ra[sa[i]] = i;

514     for (ll i = 0; i < n; i++, k -= !!k) {
199         if (ra[i] == n-1) { k = 0; continue; }
674         ll j = sa[ra[i]+1];
329         while (i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
d98         lcp[ra[i]] = k;
b76     }
5ed     return lcp;
40b }
```

4.4 Trie (Prefix Tree) 912

Stores strings over a fixed alphabet to support insert, erase, and prefix counting in linear time.

Complexity: O(|s|) per op, O(total keys)

```
ab5 struct trie {
fe0     vector<vll> to;
630     vll end, pref;
1c5     ll sigma; char norm;

a5e     trie(ll sigma_=26, char norm_='a') : sigma(sigma_), norm(
norm_) {
ce5         to = {vll(sigma)};
86e         end = {0}, pref = {0};
961     }

64e     void insert(string s) {
00d         ll x = 0;
7e7         for (auto c : s) {
00c             ll &nxt = to[x][c-norm];
dd7             if (!nxt) {
337                 nxt = sz(to);
124                 to.push_back(vll(sigma));
770                 end.push_back(0), pref.push_back(0);
6aa             }
827             x = nxt, pref[x]++;
4ab         }
421         end[x]++, pref[0]++;
7fa     }

6b2     void erase(string s) {
00d         ll x = 0;
b4f         for (char c : s) {
00c             ll &nxt = to[x][c-norm];
10c             x = nxt, pref[x]--;
d8e             if (!pref[x]) nxt = 0;
e3d         }
104         end[x]--, pref[0]--;
b69     }

680     ll find(string s) {
00d         ll x = 0;
7e7         for (auto c : s) {
2ec             x = to[x][c-norm];
59b             if (!x) return -1;
42d         }
ea5         return x;
63a     }

fde     ll count_pref(string s) {
b09         ll id = find(s);
fc1         return id >= 0 ? pref[id] : 0;
d11     }
912 };
```

4.5 Z-funtion 8ea

For any index i, finds the lcp between s[i..n) ans s O(n)

```
008 vll get_z(string s) {
3b8     ll n = sz(s);
0f9     vll z(n, 0);

837     ll l = 0, r = 0;
590     forn(i, 1, n) {
0af         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
4ed         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
65e         if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
bd8     }

070     return z;
8ea }
```

5 Dynamic Programming

5.1 Divide and Conquer DP Optimization 186

Optimizes DP transitions with quadrangle inequality/monge-like structure using divide-and-conquer over optimal decision points.

Complexity: O(K N log N) with O(1) cost, O(N)

```
66b vector<vll> dp; // dp[n+1][2]

b8e void solve(ll k, ll l, ll r, ll lk, ll rk) {
de6     if (l > r) return;
f20     ll m = (l+r)/2, p = -1;
bff     auto& ans = dp[m][k&1] = INF;
f08     for (ll i = max(m, lk); i <= rk; i++) {
d73         ll at = dp[i+1][~k&1] + cost(m, i);
57d         if (at < ans) ans = at, p = i;
d63     }
1ee     solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
35b }

c5c ll dnc(ll n, ll k) {
321     dp[n][0] = dp[n][1] = 0;
390     forn(i,0,n) dp[i][0] = INF;
050     forn(i,1,k+1) solve(i, 0, n-i, 0, n-i);
8e7     return dp[0][k&1];
40f }
```

5.2 Sum over Subsets DP (SOS DP) 264

sos_dp_sub: for each mask, computes the sum of f[s] over all subsets s of mask (i.e., f'[mask] = sum of f[s] for s in mask). sos_dp_super: for each mask, computes the sum of f[s] over all supersets s of mask (i.e., f'[mask] = sum of f[s] for mask in s). input size must be a power of two (2^n).

Complexity: O(n * 2^n), O(2^n) mem

```
9e8 vll sos_dp_sub(vll f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));

4cd     forn(i,0,n) forn(mask, 0, (1<<n))
796         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
        // -= for inverse
abe     return f;
7c3 }

6b6 vll sos_dp_super(vll f) {
f5d     ll n = __builtin_ctzll(sz(f));
310     assert((1ll<<n) == sz(f));

4cd     forn(i,0,n) forn(mask, 0, (1<<n))
a3c         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
        // -= for inverse
abe     return f;
d18 }
```

6 Computational Geometry

6.1 Circumcircle and Circumradius 643

ccRadius: calcula o circumraio funciona com pt de int ccCentar: calcula o circumcentro funciona somente com pt de double

```
f95 ld ccRadius(pt a, pt b, pt c) {
001     ld s = sarea(a,b,c);
79e     if(s == 0) return DINF;
07c     return dist(a,b)*dist(b,c)*dist(c,a)/(4*abs(s));
c59 }

209 pt ccCenter(pt a, pt b, pt c) {
bdc     c = c-a, b = b-a;
846     if((b^c) == 0) return {DINF,DINF};
c50     return a + perp((b*norm_sq(c)-c*norm_sq(b)))/(2*(b^c));
a80 }
```

6.2 Convex Hull (Monotone Chain) 55e

Computes the convex hull of a set of points using Andrew's monotone chain; handles collinear points based on ccw condition.

Complexity: O(N log N), O(N)

```
eb2 vector<pt> convex_hull(vector<pt> &v) {
d76     v.erase(unique(v.begin(), v.end()), v.end());
c2b     if(sz(v) <=1) return v;
526     vector<pt> l, u;
6fe     forn(i, 0, sz(v)) {
e5c         while(sz(l) > 1 && !ccw(l.end()[-2], l.end()[-1], v[i]))
364             l.pop_back();
c35         l.push_back(v[i]);
d19     }
a62     for(ll i = sz(v)-1; i>=0; i--) {
ecb         while(sz(u) > 1 && !ccw(u.end()[-2], u.end()[-1], v[i]))
7a8             u.pop_back();
a95         u.push_back(v[i]);
d9f     }
364     l.pop_back();
7a8     u.pop_back();
82b     for(pt i : u) l.push_back(i);
792     return l;
55e }
```

6.3 Extremal Vertex in Convex Polygon b8e

Dado comparador, calcula o maior vertice nessa ordem O(log(n))

```
40e #include "src/geometry/geo_int.cpp"

758 ll extreme(vector<pt>&pol, const function<bool(pt, pt)>& cmp) {
0b3     ll n = sz(pol);
944     auto extr = [&](ll i, bool& cur_dir) {
22a         cur_dir = cmp(pol[(i+1)%n], pol[i]);
35f         return !cur_dir && !cmp(pol[(i+n-1)%n], pol[i]);
4b2     };
63d     bool last_dir, cur_dir;
a0d     if (extr(0, last_dir)) return 0;
280     ll l = 0, r = n;
ead     while (l+1 < r) {
3e3         ll m = (l+r)/2;
f29         if (extr(m, cur_dir)) return m;
44a         bool rel_dir = cmp(pol[m], pol[l]);
8ed         if ((!last_dir && cur_dir) || (last_dir == cur_dir &&
rel_dir == cur_dir)) {
8a6             l = m;
1f1             last_dir = cur_dir;
cbe         } else r = m;
e80     }
792     return l;
8e2 }
f2a ll max_dot(vector<pt>&pol, pt v) {
805     return extreme(pol, [&](pt p, pt q) { return p*v > q*v; });
1b2 }
```

```
ef2 pair<ll, ll> tangents(vector<pt>&pol, pt p) {
ffb     auto L = [&](pt q, pt r) { return ccw(p, r, q); };
8fd     auto R = [&](pt q, pt r) { return ccw(p, q, r); };
861     return {extreme(pol, L), extreme(pol, R)};
960 }
```

6.4 Float Geometry Primitives d9d

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
088 typedef long double ld;

e3b const ld DINF = 1e18;
107 const ld eps = 1e-9;
4fd const ld pi = 2*acos(0);

b32 #define sq(x) ((x)*(x))

d00 ll cmp(ld x, ld y = 0, ld tol = eps) {
938     return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
c8d }

b2a struct pt { // ponto
c1e     ld x, y;
3dd     pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
5bc     bool operator < (const pt p) const {
c4a         if (cmp(x, p.x) != 0) return cmp(x, p.x) < 0;
48b         if (cmp(y, p.y) != 0) return cmp(y, p.y) < 0;
bb3         return 0;
b2e     }
a83     bool operator == (const pt p) const {
0e5         return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
612     }
cb9
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
a24
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
4a8
    pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
a60
    pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
3b6     ld operator * (const pt p) const { return x*p.x + y*p.y; }
6df     ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
5ed     friend istream& operator >> (istream& in, pt& p) {
e37         return in >> p.x >> p.y;
e45     }
6cb };

b3a struct line { // reta
730     pt p, q;
0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
7ab };

8c5 ld norm_sq(pt p){ return p*p;}

74c ld norm(pt p){ return sqrt(norm_sq(p)); }

b57 ld dist_sq(pt p, pt q){ return norm_sq(p - q);}
```

```
b47 ld dist(pt p,pt q){ return sqrt(dist_sq(p, q));}

589 ld angle(pt v) { // angulo do vetor com o eixo x
587     ld ang = atan2(v.y, v.x);
1e3     if (cmp(ang, 0) < 0) ang += 2*pi;
19c     return ang;
2e3 }

298 ld sarea(pt p, pt q, pt r) { // area com sinal
606     return ((q-p)^(r-q))/2;
1b1 }

e32 bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
571     return cmp(sarea(p, q, r), 0) == 0;
e64 }

82b bool ccw(pt p, pt q, pt r, bool strict = true) { // true if p, q,
 r are
dc3     return cmp(sarea(p, q, r)) > 0 || (!strict && cmp(sarea(p, q,
 r)) == 0);
073 }

099 bool isinseg(pt p, line r) { // se p pertence ao seg de r
f65     pt a = r.p - p, b = r.q - p;
26e     return cmp(a ^ b) == 0 && cmp(a * b) <= 0;
fe1 }

13d ld polarea(vector<pt> v) { // area do poligono
9c5     ld ret = 0;
ced     forn(i, 0, sz(v))
6d4         ret += sarea(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
0a8 }

98d
ld get_t(pt v, line r) { // retorna t tal que t*v pertence a reta r
6ee     return (r.p^r.q) / ((r.p-r.q)^v);
a0a }

d5c pt inter(line r, line s) { // r inter s
9a6     if (cmp((r.p - r.q) ^ (s.p - s.q), 0) == 0) return pt(DINF,
DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
9be }

// Intersecções de circulos

761 pair<pt,pt> perp_pts(line l, ld d){ // retorna dois pontos r,s
tq rq e sq sao perp a pq e |rq| = |sq| = d;
33d     pt dir = l.p - l.q;
605     pt r = pt(dir.y, -dir.x);
d11     pt k = r * (d/norm(r));
268     return {l.q + k, l.q - k};
acc }

551 pair<pt,pt> circint(pt o1, ld r1, pt o2, ld r2){
78b     pt dir = o2 - o1;
697     pt in(DINF, DINF);
837     ld d = norm(dir);
    // tangent
38a     if(cmp(r1 + r2, d) == 0 || cmp(d + r2, r1) == 0 || cmp(d +
r1, r2) == 0)
490         return {o1 + dir * r1/(r1 + r2), in};
    // no inter
eb6     if(cmp(r1 + r2, d) == -1 || cmp(d + r2, r1) == -1 || cmp(d +
 r1, r2) == -1)
4ef         return {in, in};
    // two inters
48e     ld x = (d*d - r2*r2 + r1*r1)/(2*d);
ea6
    return perp_pts(line(o1, o1 + dir * x/d), sqrt(r1*r1 - x*x));
9a6 }

//interseções de circulo e reta

1fa pt foot(pt p, pt a, pt b) {
a8a     pt dir = b-a;
dad     return (dir*((p-a)*dir))/(dir*dir) + a;
84d }

9d3 pair<pt,pt> cline(pt o, double r, pt a, pt b){
e2a     pt h = foot(o, a, b);
697     pt in(DINF, DINF);
897     ld d = norm(h - o);
b54     if(cmp(d, r) == 0) return {h, in};
b31     if(cmp(d, r) == 1) return {in, in};
696     return perp_pts(line(o, h), sqrt(r*r - d*d));
3a6 }
```

6.5 Integer Geometry Primitives 3e6

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
b2a struct pt { // point
0be     ll x, y;
f6f     pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
5bc     bool operator < (const pt p) const {
95a         if (x != p.x) return x < p.x;
89c         return y < p.y;
dcd     }
a83     bool operator == (const pt p) const {
48a         return x == p.x && y == p.y;
277     }
cb9
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
a24
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
8f0     pt operator * (const ll c) const { return pt(x*c, y*c); }
60d     ll operator * (const pt p) const { return x*(ll)p.x + y*(ll)
p.y; } // dot product
d86     ll operator ^ (const pt p) const { return x*(ll)p.y - y*(ll)
p.x; } // cross product
5ed     friend istream& operator >> (istream& in, pt& p) {
e37         return in >> p.x >> p.y;
e45     }
1b5 };

b3a struct line { // line segment
730     pt p, q;

0d6     line() {}
4b8     line(pt p_, pt q_) : p(p_), q(q_) {}
7f9     bool operator < (const line l) const {
d1d         if (!(p == l.p)) return p < l.p;
d4a         return q < l.q;
2ca     }
e1c     bool operator == (const line l) const {
05e         return p == l.p && q == l.q;
b27     }
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
ab7 };

5a2 ll sarea2(pt p, pt q, pt r) { // 2 * signed area
586     return (q-p)^(r-q);
bf4 }

e58 ll norm_sq(pt p){ return p*p;}

baa ll dist_sq(pt p, pt q){ return norm_sq(p - q);}

82b bool ccw(pt p, pt q, pt r, bool strict = true) {
966
    return sarea2(p, q, r) > 0 || (!strict && sarea2(p, q, r) == 0);
40e }

c31 int quad(pt p) { // quadrant of a point
dbb     return (p.x<0)^3*(p.y<0);
fcf }

2df bool compare_angle(pt p, pt q) { // returns true if angle(p) <
angle(q)
9fc     if (quad(p) != quad(q)) return quad(p) < quad(q);
ea1     return ccw(q, pt(0, 0), p);
771 }

099 bool isinseg(pt p, line r) { // se p pertence ao seg de r
f65     pt a = r.p - p, b = r.q - p;
3e1     return (a ^ b) == 0 && (a * b) <= 0;
562 }

676 bool interseg(line r, line s) { // se o seg de r intersecta o
seg de s
739     if (isinseg(r.p, s) || isinseg(r.q, s)
bdc         || isinseg(s.p, r) || isinseg(s.q, r)) return 1;

13c     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
413            ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
d79 }

510 ll polarea(vector<pt> v) { // area do poligono
b73     ll ret = 0;
ced     forn(i, 0, sz(v))
382         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
d03     return abs(ret);
76b }

// comparator for set to perform sweep line with segments
2c4 struct cmp_sweepline {
d80     bool operator () (const line& a, const line& b) const {
        // assumes segments have p < q
191         if (a.p == b.p) return ccw(a.p, a.q, b.q);
b4b         if (a.p.x != a.q.x && (b.p.x == b.q.x || a.p.x < b.p.x))
780             return ccw(a.p, a.q, b.p);
dc0         return ccw(a.p, b.q, b.p);
187     }
21b };
```

6.6 Halfplane Intersection 667

Acha o ponto mais extremo na direção dada só entra retas nao verticais, processar as retas verticais antes e encodar essa info no minx e maxx se quiser o ponto mais acima, fazer best_dir = (0,1) dependendo do best_dir, best = (+-INF, +-INF)

```
c79 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

877 pt hpi_1d(vector<line> v, pt best_dir, ll n, line suport, ll
minx, ll maxx){
3ec     pt low = inter(line(pt(minx, 0), pt(minx,1)), suport), upp =
 inter(line(pt(maxx, 0), pt(maxx,1)), suport);
522     forn(i, 0, n){
beb         pt intersection = inter(v[i], suport);
f1c         debug(low);
dda         debug(upp);
f9d         debug(intersection);
138         debug(v[i]);

5b8         if(!ccw(v[i].p, v[i].q, low)) low = intersection;
030         else if(!ccw(v[i].p, v[i].q, upp)) upp = intersection;
9b8     }

a7b     if(cmp(low.x, upp.x) > 0) return {DINF, DINF};

5b1     if(cmp(best_dir*low, best_dir*upp) <= 0) return upp;
b2b     return low;
b43 }

219 pt hpi(vector<line> v, pt best_dir, ll minx, ll maxx){
f21     shuffle(v.begin(), v.end(), rng);

dce     ll n = sz(v);
3a3     pt best(1, 1);

522     forn(i, 0, n){
20a         debug(best);
138         debug(v[i]);
121         if(ccw(v[i].p, v[i].q, best)) continue;
186         best = hpi_1d(v, best_dir, i, v[i], minx, maxx);
ff8         assert(!(best ==pt(DINF, DINF)));
4c5     }

f26     return best;
f0a }
```

6.7 Point Inside Polygon 11a

responde se um ponto está dentro do poligono com ray casting se strict = true ele nao considera pontos na borda como interior O(n)

```
ffe #include "src/geometry/geo_double.cpp"

f40 bool inpolygon(vector<pt> &p, pt a, bool strict = true) {
974     ll cnt = 0, n = sz(p);
522     forn(i,0,n) {
61c         pt q = p[(i + 1) % n];
03a         if (isinseg(a, line(p[i], q))) return !strict;
793         cnt ^= (((cmp(a.y, p[i].y) < 0) - (cmp(a.y,q.y) < 0))*((
p[i]-a)^(q-a))) > 0 ;
dc9     }
70a     return cnt;
a17 }
```

6.8 Point Inside Convex Polygon 4a9

responde se um ponto está dentro do poligono convexo com binary search se strict = true ele nao considera pontos na borda como interior O(log(n))

```
40e #include "src/geometry/geo_int.cpp"

c20 bool inpolygonconv(vector<pt> &pol, pt p, bool strict = true) {
b6e     if (pol.size() == 0) return false;
27a     if (pol.size() == 1) return (!strict && p == pol[0]);
09b     ll l = 1, r = pol.size();
40c     while (l < r) {
3e3         ll m = (l+r)/2;
48f         if (ccw(p, pol[0], pol[m])) l = m+1;
ef3         else r = m;
206     }
baa     if (l == 1) return (!strict && isinseg(p, line(pol[0], pol
[1])));
9e7     if (l == pol.size()) return false;
d57     return !ccw(p, pol[l], pol[l-1], !strict);
100 }
```

6.9 Minimum Enclosing Circle b3c

calcula o minimum enclosing circle, probalistico mas e O(n) so se o pt for de double!! talvez usar r(1+eps);*

```
faf pair<pt, ld> mec(vector<pt> v) {
7c9     shuffle(v.begin(), v.end(), mt19937(time(0)));
d91     pt o = v[0];
e1e     ld r = 0;
eb8     forn(i, 0, sz(v)) if (cmp(dist(o, v[i]),r) > 0) {
983         o = v[i], r = 0;
141         forn(j, 0, i) if (cmp(dist(o, v[j]),r) > 0) {
76e             o = (v[i] + v[j]) / 2;
f66             r = dist(o, v[i]);
21c             forn(k, 0, j) if (cmp(dist(o, v[k]),r) > 0) {
6b1                 o = ccCenter(v[i], v[j], v[k]);
f66                 r = dist(o, v[i]);
8d3             }
c74         }
200     }
645     return {o, r};
b3c }
```

6.10 Minkowski Sum 374

Soma poligonos de tamanho a e b em O(a+b)

```
40e #include "src/geometry/geo_int.cpp"

539 vector<pt> minkowski(vector<pt> p, vector<pt> q) {
051     auto fix = [](vector<pt>& P) {
515
        rotate(P.begin(), min_element(P.begin(), P.end()), P.end());
018         P.push_back(P[0]), P.push_back(P[1]);
f24     };
889     fix(p), fix(q);
8af     vector<pt> ret;
692     int i = 0, j = 0;
2ee     while (i < p.size()-2 or j < q.size()-2) {
898         ret.push_back(p[i] + q[j]);
732         auto c = ((p[i+1] - p[i]) ^ (q[j+1] - q[j]));
ebc         if (c >= 0) i = min<int>(i+1, p.size()-2);
81e         if (c <= 0) j = min<int>(j+1, q.size()-2);
9ff     }
edf     return ret;
d7c }
```

6.11 Plota Poligonos 73d

Ajuda a debugar codigo de geometria

```
def svg(P, o="d.svg"):
    if not P: return
    X, Y = zip(*P)
    x0, x1, y0, y1 = min(X), max(X), min(Y), max(Y)
    p = max(x1-x0, y1-y0) * 0.1 or 10
    head = f'<svg viewBox="{x0-p} {y0-p} {x1-x0+2*p} {y1-y0+2*p}"
xmlns="http://www.w3.org/2000/svg" style="background:#eee">'
    poly = f'<polygon points="{" ".join(f"{x},{y}" for x,y in P)}"
fill="#f003" stroke="red" stroke-width="{p/20}"/>'
    text = "".join([f'<text x="{x}" y="{y}" font-size="{p/3}">{i}</
text>' for i,(x,y) in enumerate(P)])
    with open(o, 'w') as f: f.write(head + poly + text + '</svg>')
```

6.12 Polygon Cut 00d

Corta um poligono deixando somente a parte ccw com a reta l dada O(n)

```
39e vector<pt> polygonCut(const vector<pt>& poly, line l) {
819     vector<pt> res;
e29     forn(i,0,sz(poly)) {
07c         pt curr = poly[i], prev = i ? poly[i-1] : poly.back();
dc8         bool a = ccw(l.p, l.q, curr), b = ccw(l.p, l.q, prev);
123         if (a != b)
1e1             res.push_back(inter(line(curr, prev), l));
754         if (a)
fb3             res.push_back(curr);
c0d     }
b50     return res;
00d }
```

6.13 Segment Sweep Line Skeleton def

Maintains an active set of segments ordered for sweep-line processing over x; insertion and removal are typically logarithmic.

```
719 map<ll, set<line, cmp_sweepline>> sweepline_begin; // dado um x,
 diz quais linhas comecam naquele x
5a6 map<ll, set<line, cmp_sweepline>> sweepline_end;   // dado um x,
 diz quais linhas terminam naquele x

765 void process_beg(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line, vll& parent){
47d     for(auto x : v){
380         active_line.insert(x);
         // processar uma linha que esta sendo adicionada
9c5     }
fb4 }

923 void process_end(set<line, cmp_sweepline>& v, set<line,
cmp_sweepline>& active_line){
47d     for(auto x : v){
d76         active_line.erase(x);
a1d     }
68a }

ec5 void sweepline(ll n){
23e     set<line, cmp_sweepline> active_line;

521     while(!sweepline_begin.empty() || !sweepline_end.empty()){
```

```
c58         auto it_beg = sweepline_begin.begin();
aa0         auto it_end = sweepline_end.begin();

385         if(sweepline_end.empty()){
570             process_beg(it_beg->second, active_line, parent);
7ae             sweepline_begin.erase(it_beg);
5e2             continue;
a8b         }

379         if(sweepline_begin.empty() || it_end->first <= it_beg->
first){
2a4             process_end(it_end->second, active_line);
61a             sweepline_end.erase(it_end);
5e2             continue;
ef4         }

570         process_beg(it_beg->second, active_line, parent);
7ae         sweepline_begin.erase(it_beg);
b06     }
76d }
```

7 Extra (Templates and Utilities)

7.1 bash_alias b5d

```
c76 echo "alias make-debug='make CPPFLAGS=\"-DDEBUG\"'" >> ~/.bashrc
ad1 source ~/.bashrc
```

7.2 1D Coordinate Compression 193

Maps large or sparse values to a dense range [0, N-1]. Uses sort, unique, and lower_bound for fast mapping. Time Complexity: O(N log N) | Space Complexity: O(N)

```
8f1 void compress(){
0d8     vll vals;
3d9     sort(vals.begin(), vals.end());
7ee     vals.erase(unique(vals.begin(), vals.end()), vals.end());

aea     auto compress = [&](ll x) {
b03         return (ll)(lower_bound(vals.begin(), vals.end(), x) -
vals.begin());
823     };
193 }
```

7.3 hash 826

```
sed -n $2',’$3’ p’ $1 | sed ’/^#w/d’ | cpp -dD -P -fpreprocessed |
tr -d ’[:space:]’ | md5sum | cut -c-3
```

7.4 makefile 885

```
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g -
Wall -Wshadow -std=c++20 -Wno-unused-result -Wno-sign-compare
```

7.5 Otimizacoes agressivas, pode deixar mais rapido ou mais devagar 35c

```
4fd #pragma GCC optimize("Ofast")
// Soft Ofast
73e #pragma GCC optimize("O3")
// Auto explicativo
f00 #pragma GCC optimize("unroll-loops")
// Vetorizacao
ee8 #pragma GCC target("avx2")
// Para operacoes com bits
e15 #pragma GCC target("bmi,bmi2,popcnt,lzcnt")
// tem que juntar os target na mesma linha
```

7.6 random b0e

```
c79 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

f89 ll uniform(ll l, ll r){
969     uniform_int_distribution<ll> uid(l, r);
f54     return uid(rng);
cac }
```

7.7 stress d5d

```
P=a
make ${P} ${P}2 gen || exit 1
for ((i = 1; ; i++)) do
    ./gen $i > tin
    ./${P} < tin > tout
    ./${P}2 < tin > tout2
    if (! cmp -s tout tout2) then
        echo "--> entrada:"
        cat tin
        echo "--> saida1:"
        cat tout
        echo "--> saida2:"
        cat tout2
        break;
    fi
    echo $i
done
```

7.8 template fa9

```
2b7 #include <bits/stdc++.h>
ca4 using namespace std;

ad1 typedef long long ll;
0d2 typedef pair<ll, ll> pll;
3a6 typedef vector<ll> vll;

4d1 #define forn(i, s, e) for (ll i = (s); i < (e); i++)
66d #define sz(u) ((ll) u.size())
cb9 #define ln "\n"

ba0 #ifdef DEBUG
2d9 #define trace(u) u
931 #define _
8c1 #else
56a #define trace(u)
aef #define _ ios::sync_with_stdio(0); cin.tie(0)
f2e #endif

721 #define debug(u) trace(cout << #u " = " << u << ln)
e98 #define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout
<< xx << " "; cout << ln)

563 const ll INF = 0x3f3f3f3f3f3f3f3fll;

// int main() {
//     _;
//     return 0;
// }
```

7.9 tester 155

```
P=a
make "$P" || exit 1

for i in {1..9}; do
    ./"$P" < "in/${P}${i}" > out_ &&
    cmp -s out_ "out/${P}${i}" &&
    echo "OK $i" ||
    { echo "NOT $i"; break; }
done
```

1 Counting Problems

1.1 Burnside's Lemma

Let G be a group that acts on a set X. The Burnside Lemma states that the number of distinct orbits is equal to the average number of points fixed by an element of G.

$$T = \frac{1}{|G|} \sum_{g \in G} |\texttt{fix}(g)|$$

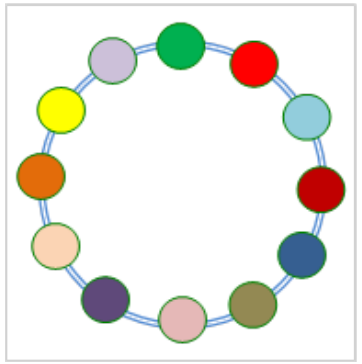
Where a orbit $\texttt{orb}(x)$ is defined as

$$\texttt{orb}(x) = \{y \in X : \exists g \in G \; gx = y\}$$

and $\texttt{fix}(g)$ is the set of elements in X fixed by g

$$\texttt{fix}(g) = \{x \in X : gx = x\}$$

Example: With k distinct types of beads how many distinct necklaces of size n can be made? Considering that two necklaces are equal if the rotation of one gives the other.



$$T = \frac{1}{n} \sum_{i=0}^{n-1} k^{gcd(i,n)}$$

1.2 Catalan Numbers

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452, 18367353072152, 69533550916004, 263747951750360, 1002242216651368.

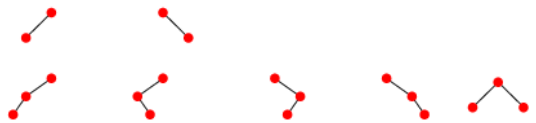
$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \prod_{k=2}^{n}\frac{n+k}{k}, \quad n \geq 0$$

Applications:

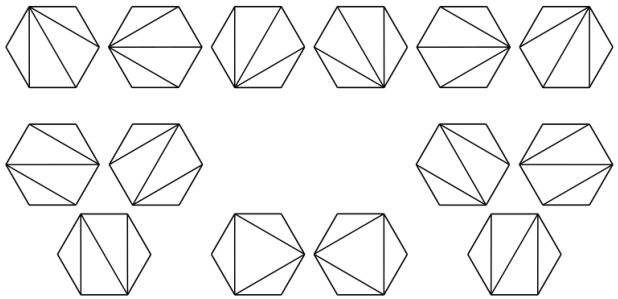
- C_n counts the number of expressions containing n pairs of parentheses which are correctly matched.

  (((()))) ()(()) ()()() (())() ...

- Successive applications of a binary operator can be represented in terms of a full binary tree. (A rooted binary tree is full if every vertex has either two children or no children.) It follows that C_n is the number of full binary trees with n + 1 leaves:



- C_n is the number of different ways a convex polygon with n + 2 sides can be cut into triangles by connecting vertices with straight lines (a form of Polygon triangulation). The following hexagons illustrate the case n = 4:



2 Graphs

2.1 Sum of Subtrees of a Tree

For a rooted tree T with n vertices, let $sz(v)$ be the size of the subtree of v. Then the following holds:

$$\sum_{v \in V}\left[\text{sz}(v) + \sum_{u \text{ child of } v}\text{sz}(u)(\text{sz}(v) - \text{sz}(u))\right] = n^2$$

3 Number Theory

3.1 Chicken McNugget Theorem

Given two **coprime** numbers n, m, the largest number that cannot be written as a linear combination of them is $nm - n - m$.

- There are $\frac{(n-1)(m-1)}{2}$ non-negative integers that cannot be written as a linear combination of n and m;
- For each pair $(k, nm - n - m - k)$, for $k \geq 0$, exactly one can be written.

3.2 Identities

$$\sum_{d|n}\varphi(d) = n$$

$$\sum_{\substack{i<n \\ \gcd(i,n)=1}} i = n \cdot \frac{\varphi(n)}{2}$$

$$|\{(x,y) : 1 \leq x,y \leq n,\ \gcd(x,y) = 1\}| = \sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor^2$$

$$\sum_{x=1}^{n}\sum_{y=1}^{n}\gcd(x,y) = \sum_{k=1}^{n}k\sum_{k|l}^{n}\left\lfloor\frac{n}{l}\right\rfloor^2\mu\left(\frac{l}{k}\right)$$

$$\sum_{x=1}^{n}\sum_{y=x}^{n}\gcd(x,y) = \sum_{g=1}^{n}\sum_{g|d}^{n}g\cdot\varphi\left(\frac{d}{g}\right)$$

$$\sum_{x=1}^{n}\sum_{y=1}^{n}\text{lcm}(x,y) = \sum_{d=1}^{n}d\,\mu(d)\sum_{d|l}^{n}l\binom{\lfloor\frac{n}{l}\rfloor+1}{2}^2$$

$$\sum_{x=1}^{n}\sum_{y=x+1}^{n}\text{lcm}(x,y) = \sum_{g=1}^{n}\sum_{g|d}^{n}d\cdot\varphi\left(\frac{d}{g}\right)\cdot\frac{d}{g}\cdot\frac{1}{2}$$

$$\sum_{x\in A}\sum_{y\in A}\gcd(x,y) = \sum_{t=1}^{n}\left(\sum_{l|t}\frac{t}{l}\mu(l)\right)\left(\sum_{t|a}\text{freq}[a]\right)^2$$

$$\sum_{x\in A}\sum_{y\in A}\text{lcm}(x,y) = \sum_{t=1}^{n}\left(\sum_{l|t}\frac{l}{t}\mu(l)\right)\left(\sum_{a\in A,\,t|a}a\right)^2$$

3.3 Large Prime Gaps

For numbers until 10^9 the largest gap is 400.
For numbers until 10^{18} the largest gap is 1500.

3.4 Lucas's Theorem

$$\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$$

For p prime. n_i and m_i are the coefficients of the representations of n and m in base p.

3.5 Mobius Inversion

If $g(n) = \sum_{d|n}f(d)$, then $f(n) = \sum_{d|n}g(d)\mu(\frac{n}{d})$.
A more useful definition is: $\sum_{d|n}\mu(d) = [n = 1]$
Example, sum of LCM:

$$\begin{aligned}
\sum_{i=1}^{n}\sum_{j=1}^{n}\text{lcm}(i,j) &= \sum_{k=1}^{n}\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=k]\frac{ij}{k} \\
&= \sum_{k=1}^{n}\sum_{a=1}^{\lfloor\frac{n}{k}\rfloor}\sum_{b=1}^{\lfloor\frac{n}{k}\rfloor}[\gcd(a,b)=1]abk \\
&= \sum_{k=1}^{n}k\sum_{a=1}^{\lfloor\frac{n}{k}\rfloor}a\sum_{b=1}^{\lfloor\frac{n}{k}\rfloor}b\sum_{d=1}^{\lfloor\frac{n}{k}\rfloor}[d|a]\,[d|b]\,\mu(d) \\
&= \sum_{k=1}^{n}k\sum_{d=1}^{\lfloor\frac{n}{k}\rfloor}\mu(d)\left(\sum_{a=1}^{\lfloor\frac{n}{k}\rfloor}[d|a]\,a\right)\left(\sum_{b=1}^{\lfloor\frac{n}{k}\rfloor}[d|b]\,b\right) \\
&= \sum_{k=1}^{n}k\sum_{d=1}^{\lfloor\frac{n}{k}\rfloor}\mu(d)\left(\sum_{p=1}^{\lfloor\frac{n}{kd}\rfloor}p\right)\left(\sum_{q=1}^{\lfloor\frac{n}{kd}\rfloor}q\right)
\end{aligned}$$

3.6 Number of Divisors

n	6	60	360	5040	55440	720720	4324320	21621600
$d(n)$	4	12	24	60	120	240	384	576

n	367567200	6983776800	13967553600	321253732800
$d(n)$	1152	2304	2688	5376

$1840105593812566080 0 \approx 2e18$ is highly composite with 184320 divisors.
For numbers up to 10^{88}, $d(n) < 3.6\sqrt[3]{n}$.

3.7 Prime Counting Function - $\pi(x)$

The prime counting function is asymptotic to $\frac{x}{\log x}$, by the prime number theorem.

x	10	10^2	10^3	10^4	10^5	10^6	10^7	10^8
$\pi(x)$	4	25	168	1 229	9 592	78 498	664 579	5 761 455

3.8 Some Primes

999999937 1000000007 1000000009 1000000021 1000000033

$10^{18} - 11$ $10^{18} + 3$ $2305843009213693951 = 2^{61} - 1$